

Automatic Image Analysis Script

September 21, 2026

Contents

1	Introduction to Automatic Image Analysis	2
1.1	Lecture Aim and the Inverse Problem	2
1.2	From Pixels to Interpretation	3
1.3	Data Types and Interpretation Tasks	5
1.4	The Classical Processing Chain	6
1.5	Models, Data, and Evaluation	8
1.6	Topic Roadmap	9
2	Neural Networks and Backpropagation	10
2.1	From Perceptrons to Multilayer Networks	10
2.2	Losses, Optimization, and the Need for Gradients	12
2.3	Backpropagation	13
2.4	Automatic Differentiation and Practical Training	15
2.5	Generalization: Overfitting, Bias, Variance, and Regularization	15
3	Visual Object Models	16
3.1	Detecting Lines: The Hough Transform	17
3.2	Detecting Arbitrary Shapes: The Generalized Hough Transform	21
3.3	Where to Look: Salient Regions	24
3.4	What to Record: Local Features and Shape Descriptors	27
3.5	Putting It Together: The Implicit Shape Model	28
A	Figures: Visual Object Models	30
B	Edge Detection Essentials	35
C	Fourier Transform and Fourier Descriptors	35
C.1	The Discrete Fourier Transform	35
C.2	Fourier Descriptors	37
D	Classical Classifiers and Part-Based Detectors	38

1 Introduction to Automatic Image Analysis

Automatic image analysis starts from the central difficulty of vision, then moves through the main processing levels, the recurring goals of the field, and the practical question of how to evaluate whether a vision system is actually reliable.

1.1 Lecture Aim and the Inverse Problem

Automatic image analysis is the problem of turning measurements into explanations. A digital image is easy to store: it is a grid of values. It is much harder to understand: we want to infer objects, surfaces, motion, geometry, lighting, and meaning from those values.

Computer vision is the computational study of how useful descriptions of the visual world can be recovered from images and video. The description may be semantic, such as “there is a pedestrian in front of the car”; geometric, such as “this surface is two meters away”; or operational, such as “reject this part because it contains a defect”. This definition is deliberately broader than object recognition. A vision system may label an image, reconstruct a three-dimensional scene, align several images, estimate motion, improve an image for a human observer, or supply measurements for a larger decision-making system.

Computer vision is therefore not just image enhancement. It is inference under uncertainty. The image is evidence; the scene that produced it is hidden.

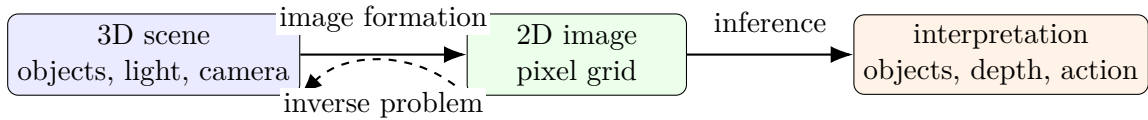


Figure 1: Computer vision tries to invert the image formation process: pixels are observed, but the scene explanation is hidden.

Human vision feels immediate, but this ease is misleading. A camera records a two-dimensional projection of a three-dimensional world. During that projection, many causes are mixed together: object shape, material, illumination, viewpoint, occlusion, sensor noise, and motion. Different scenes can produce very similar images, so there is usually no unique answer that can be read directly from the pixels.

This makes vision an inverse problem. The forward direction is comparatively clear: if we know the scene, the light, the camera, and the materials, we can model how an image is formed. The inverse direction asks us to go backward from image measurements to a plausible explanation of the world. That backward step requires assumptions, models, data, and algorithms.

A compact way to write the forward model is

$$I(x, y) = f(\Pi(X, Y, Z), L, M, C) + \varepsilon, \quad (1)$$

where $I(x, y)$ is the measured pixel value, Π is the camera projection from 3D to 2D, L describes illumination, M material and surface properties, C camera response, and ε sensor noise. Vision asks for some hidden scene state S from the image I :

$$\hat{S} = \arg \max_S p(S | I) = \arg \max_S p(I | S) p(S). \quad (2)$$

The likelihood $p(I | S)$ says which images a scene would probably produce. The prior $p(S)$ encodes what kinds of scenes are plausible before seeing the image.

The gap between the pixel array and the human interpretation is the essential problem. A person immediately infers objects, depth, material, intent, and context, while the measured image itself only contains sampled intensities or colors.

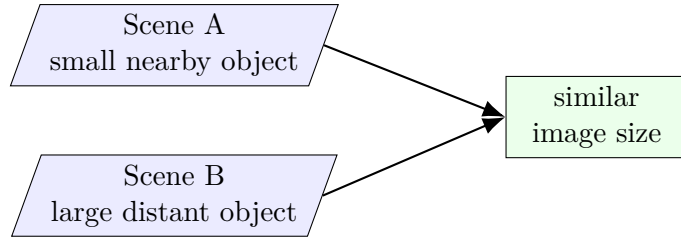


Figure 2: Ambiguity is unavoidable: different scenes can lead to similar image measurements. Priors and additional evidence help choose an explanation.

1.2 From Pixels to Interpretation

Automatic image analysis can be organized as a progression from low-level to high-level tasks.

- Low-level processing improves or transforms image measurements, for example acquisition, sampling, denoising, filtering, contrast changes, and edge extraction. Its typical mapping is image to image, or scene to image when the acquisition process is included.
- Mid-level analysis groups measurements into more meaningful structures, such as regions, contours, features, segments, motion fields, or depth estimates. It is often the bridge from image measurements to feature descriptions.
- High-level interpretation assigns semantic meaning, such as recognizing objects, estimating scene layout, detecting abnormalities, or tracking actions. Its typical mapping is from features or image evidence to object models, object detections, categories, or decisions.

These levels are useful for teaching, but real systems often mix them. A modern recognition model may learn low-level filters and high-level categories inside one network. A classical pipeline may use carefully designed features followed by statistical inference. In both cases, the goal is the same: reduce the ambiguity of image data until a useful decision or description becomes possible.

Level	Typical mapping	Typical question	Examples
Low-level	scene/image → image	What local measurements are reliable?	denoising, gradients, edges
Mid-level	image → features	Which measurements belong together?	regions, contours, segments, motion
High-level	features/image → objects	What does the image mean?	objects, scene layout, actions

Table 1: A teaching taxonomy for image-analysis tasks.

The same hierarchy can also be read as a contrast between bottom-up and top-down processing. A bottom-up system starts with the image data and builds larger hypotheses from local evidence: pixels become edges, edges become contours, contours become regions, and regions become object candidates. This is data-driven and attractive because the image itself constrains the result. Its weakness is that local evidence is often ambiguous or missing.

Top-down processing starts from a model, context, task goal, or prior expectation and asks how the image should be interpreted under that hypothesis. This is knowledge-driven: a rough object model can guide the search for parts, a scene context can make one interpretation more plausible than another, and a classifier can project learned category knowledge onto the image. Top-down processing is powerful because it uses what is already known, but it can also produce confident misinterpretations when the prior is wrong. Practical systems therefore often use a

mixed mode: bottom-up evidence proposes or constrains hypotheses, while top-down models select, refine, or reject them.

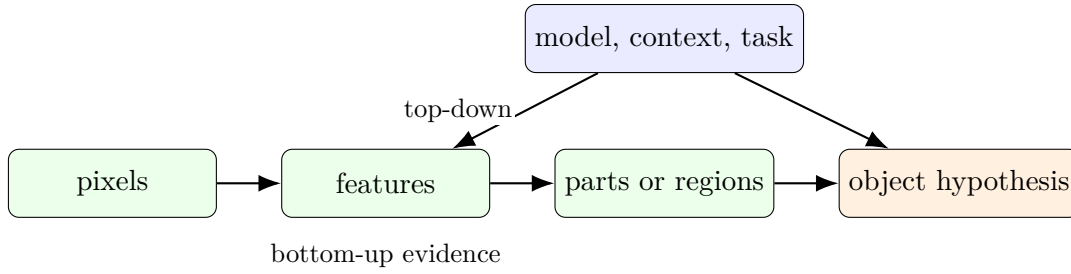


Figure 3: Bottom-up processing builds interpretations from image evidence. Top-down processing uses models and context to guide or correct the interpretation.

Computer vision can be organized around three recurring goals: recognition, reconstruction, and reorganization.

Recognition. Recognition asks what is present and where it is. Examples include classification, detection, segmentation, reading text, identifying faces, or finding defects in industrial inspection.

Reconstruction. Reconstruction asks what spatial or physical structure produced the image. Examples include estimating depth, camera motion, three-dimensional shape, or the geometry of a scene from one or more images.

Reorganization. Reorganization changes the image representation so that later analysis becomes easier or the image becomes more useful to a person. Examples include registration, stitching, stabilization, denoising, super-resolution, and computational photography.

These goals are connected. Recognition benefits from good representations; reconstruction needs reliable correspondences; reorganization often depends on understanding motion, geometry, or image formation.

Area	Main transformation	Typical examples
Digital image processing	image $\rightarrow$ image	enhancement, restoration, filtering, compression
Photogrammetry and geometry	image(s) $\rightarrow$ 3D model	camera calibration, depth, surface reconstruction, mapping
Automatic image analysis	image $\rightarrow$ object model or detection	feature extraction, segmentation, object categorization, recognition

Table 2: Automatic image analysis is closest to the recognition branch of computer vision: it studies how image evidence becomes object-level descriptions, while still using processing and geometric ideas when they support that goal.

Another useful taxonomy separates the vocabulary of the field by the kind of question being asked. Some terms describe the *input* and its formation, some describe *representations* computed from the image, some describe *tasks*, and some describe the *knowledge* used to resolve ambiguity. Keeping these groups separate helps avoid a common beginner confusion: a feature, a model, a task, and an application are not the same kind of object.

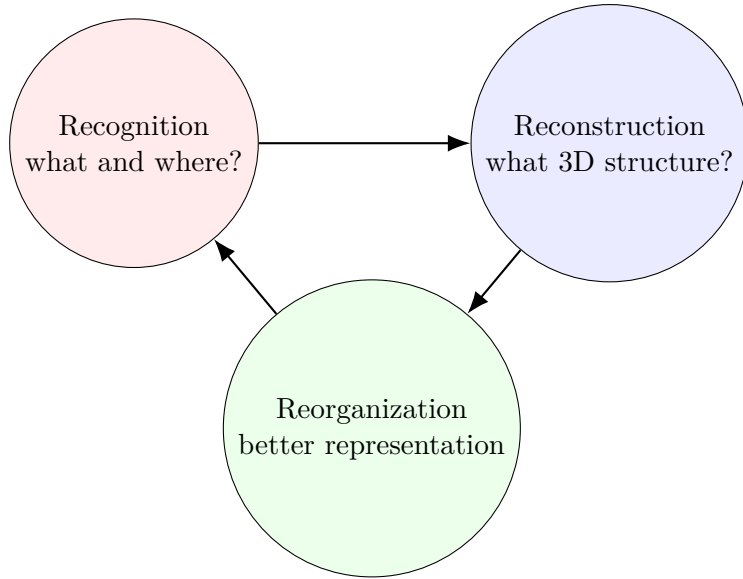


Figure 4: Recognition, reconstruction, and reorganization are not isolated topics; each goal supplies information that can support the others.

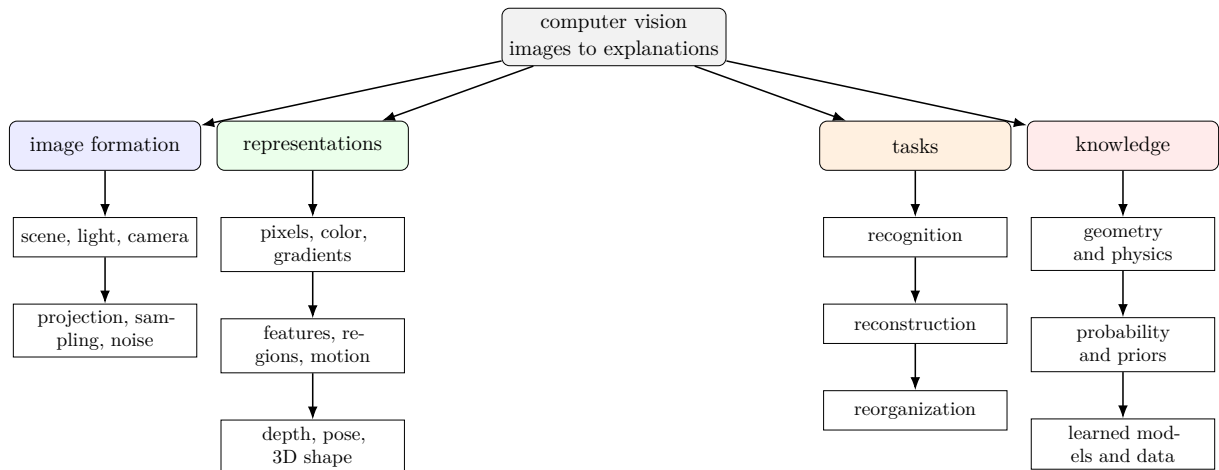


Figure 5: A compact taxonomy of introductory computer-vision terms. The field connects image formation, intermediate representations, task goals, and prior knowledge.

1.3 Data Types and Interpretation Tasks

Before listing concrete tasks, it helps to be precise about what kind of data an image is. We distinguish *structured* from *unstructured* data by whether the arrangement of the values carries meaning. A list of attributes such as (70 kg, 170 cm, 32 years) is unstructured: the entries can be permuted without changing what is described, as long as we remember which field is which. An image is structured, because the spatial layout of the pixels *is* the information. Shuffling the pixels destroys the content even though the multiset of intensity values is unchanged; sound and text are structured for the same reason. This is exactly why image analysis cannot treat a picture as an unordered bag of intensities: position and neighborhood are part of the signal, and the methods in later sections are built to exploit them.

High-level interpretation is not one task but a family of related questions, and it is worth naming them precisely because they differ in what the output looks like. *Image classification* assigns the whole image, or a given region, to one of a fixed set of categories, without saying where anything is. *Object detection* adds localization: it returns a set of objects, each with a

position, usually a bounding box, together with a category, so it answers “what” and “where” at once. *Semantic segmentation* pushes localization to the pixel level, assigning a class label to every pixel and thereby partitioning the image into semantically uniform regions, though without separating individual instances of the same class. *Object recognition* is the broader goal of identifying which objects are present; in its narrow sense it means recognizing a specific identity or instance, such as a particular face or landmark, and in its broad sense it is used as an umbrella over classification and detection. Beyond these, interpretation also includes pose estimation, which recovers the spatial configuration of an object, action recognition, which labels an activity in a video, and attribute classification, which describes properties rather than category.

Task	Output	Question it answers
Image classification	one category for the whole image or region	what is in this image?
Object detection	set of located objects, e.g. boxes plus labels	what is present and where?
Semantic segmentation	a class label for every pixel	which class does each pixel belong to?
Object recognition	identity or category of the objects present	what object is this?
Pose estimation	spatial configuration, e.g. key-points or orientation	how is the object arranged?
Action recognition	activity label for a clip	what is happening over time?

Table 3: High-level interpretation tasks differ mainly in how much localization they require, from a single image-level label up to a decision at every pixel.

Classifying a real application means asking what the system has to output. A building access system that shows a face to a camera and matches it against a database of employee faces must decide *which* person this is: that is object recognition in the narrow, identity sense. It needs no pixel labels (segmentation), no free-text description (captioning), and localization is only a helper step. A system that analyzes basketball games, draws boxes around players, reads their names and numbers, and states whether a player is dribbling, throwing, or defending combines several tasks: the boxes and identities are object detection and recognition, and the verbs describe activities over time, which is action recognition. Exam questions of this kind reward naming *every* task whose output the system produces.

1.4 The Classical Processing Chain

All of these tasks share the same underlying processing chain. Image acquisition and preprocessing produce reliable low-level measurements, mid-level analysis groups them into features, and the high-level stage turns those into the task-specific output. In classical image analysis every stage is a separate, hand-designed module, and the order of the stages is fixed: Figure 6 shows it and Table 4 gives one example per stage.

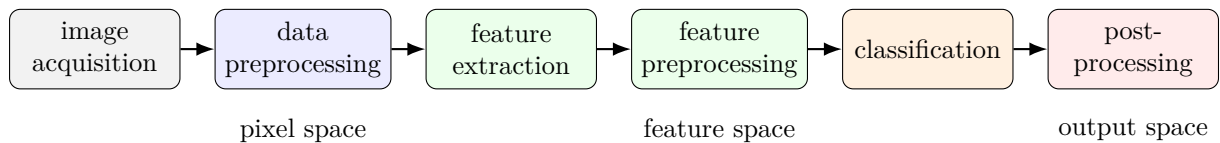


Figure 6: The classical image-analysis processing chain. Data preprocessing stays in pixel space, feature extraction maps pixels to a feature space, and only the classifier maps features to the output space of labels.

Stage	Mapping	Typical example
Image acquisition	scene $\rightarrow$ image	camera, sensor, sampling
Data preprocessing	image $\rightarrow$ image	denoising, histogram equalization to remove illumination differences
Feature extraction	image $\rightarrow$ feature vector	intensity histogram, HOG, bag of visual words
Feature preprocessing	features $\rightarrow$ features	normalization, feature selection, dimensionality reduction
Classification	features $\rightarrow$ label	linear classifier, SVM, random forest, MLP
Postprocessing	labels $\rightarrow$ labels	smoothing label maps, non-maxima suppression of detections

Table 4: One example per stage of the processing chain. The stage is decided by what goes in and what comes out: an operation that returns an image is preprocessing, one that returns a feature vector is feature extraction.

The mapping column is the tool for sorting any method into its stage. Applying a transformation so that all images of a dataset have approximately the same histogram changes pixel values but still outputs images, so it is *data* preprocessing, not feature extraction. Feature extraction maps from pixel space into a *feature* space, not directly into the output space; turning features into labels is the classifier’s job.

Two classical features. The simplest image feature is the *intensity histogram*: count how many pixels take each grey value and normalize the counts. The histogram ignores where the pixels are, so rotating (or mirroring, or shuffling) the image leaves it unchanged; it is rotation invariant. It is *not* illumination invariant, because brightening the scene shifts all values and therefore the histogram. The *bag of visual words* (BoVW) transfers the same counting idea from grey values to image parts:

1. detect keypoints in all training images and describe each by a local descriptor;
2. cluster the descriptors (e.g. with k -means); each cluster center is a *visual word*, and together they form the vocabulary;
3. for an image, assign every keypoint to its nearest word and count how often each word occurs, which gives a word histogram;
4. compare this histogram with those of the database images, or feed it to a classifier.

Like the intensity histogram, BoVW records *which* parts occur, but not *where*: it describes no spatial relations between the patches. Part-based models that keep the geometry, such as the implicit shape model of Section 3, replace the plain count by votes with offset vectors.

From classifier to detector. A classifier trained on image patches of a fixed size answers only “is this patch an object?” Turning it into a detector requires three additions. First, a *sliding window* applies the classifier at every position of the image. Second, an *image pyramid* rescales the image to several resolutions so that objects of different sizes fit the fixed window. Third, each positive response gives a bounding box, whose size follows from the pyramid level and whose position follows from the window location; overlapping boxes around the same object are then reduced to the strongest one by *non-maxima suppression*.

Where deep learning fits. Deep networks do not remove the stages but merge them: a single differentiable model can learn preprocessing, feature extraction, feature selection, and classification jointly from data. A neural network can therefore play any of these roles alone, as a learned feature extractor whose activations feed another classifier, as a feature selector, or as the classifier on top of hand-crafted features.

1.5 Models, Data, and Evaluation

Because images are ambiguous, vision systems need prior knowledge. Sometimes that knowledge is explicit: camera geometry, physical image formation, smooth surfaces, rigid motion, or probabilistic noise models. Sometimes it is learned from data: neural networks discover features and decision rules from many examples.

Classical computer vision often emphasizes explicit models and engineered features. Modern deep learning often emphasizes learned representations and large datasets. The practical lesson is not that one replaces the other completely. Robust systems usually combine several kinds of knowledge: scientific models for image formation, statistical models for uncertainty, engineering choices for reliability, and data-driven learning for complex patterns that are hard to specify by hand.

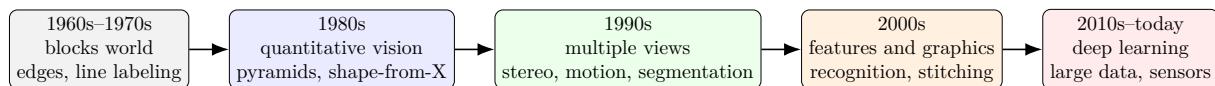


Figure 7: A brief historical timeline. The emphasis moved from hand-built geometric reasoning, through quantitative reconstruction and engineered features, toward large-scale learned representations and integrated sensing.

The timeline is not a story of one idea simply replacing another. Edge operators, camera geometry, variational estimation, probabilistic models, feature matching, and neural networks all remain useful. What changes over time is the balance between explicit modeling, available computation, available training data, and the kinds of applications that can be deployed reliably.

Source of knowledge	What it contributes	Risk if missing
Physical model	Camera geometry, illumination, projection	Results may violate how images are formed
Statistical model	Noise, uncertainty, priors, inference	The system may overtrust ambiguous measurements
Engineering design	Robustness, speed, failure handling	The method may work only in demonstrations
Data-driven learning	Features and decision rules from examples	Complex visual variation may be underspecified

Table 5: A practical vision system usually combines several kinds of knowledge.

Computer vision is now a basic technology in many domains. It is more useful to name the application categories than to memorize a long list of products: consumer imaging and media, medicine and life sciences, industry and logistics, robotics and autonomous systems, remote sensing and mapping, document analysis, security and biometrics, and scientific measurement. The tasks differ, but the same conceptual questions recur: what image evidence is available, what assumptions make the interpretation possible, and what failure cases matter in the application context?

Application category	Typical vision output	Main reliability concern
Consumer and media	enhanced images, alignment, tracking	visual quality and robustness across devices
Medicine and life sciences	measurements, segmentations, suspected findings	uncertainty, bias, and clinical validation
Industry and logistics	defects, codes, counts, object poses	speed, repeatability, and rare failures
Robotics and autonomy	localization, obstacles, maps, actions	safety under changing environments
Remote sensing and mapping	land cover, 3D structure, change detection	scale, resolution, and registration accuracy
Documents and biometrics	text, identity cues, authenticity signals	privacy, fairness, and adversarial misuse

Table 6: Categorical application areas are easier to study than isolated product examples because they expose recurring reliability questions.

These applications also show why evaluation matters. A result that looks plausible on one example is not enough. Vision algorithms must be tested against noise, changing illumination, clutter, unusual viewpoints, occlusion, and data that differ from the training or development examples.

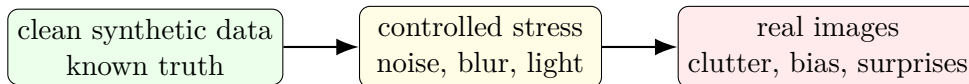


Figure 8: Evaluation should move from controlled correctness toward real-world robustness.

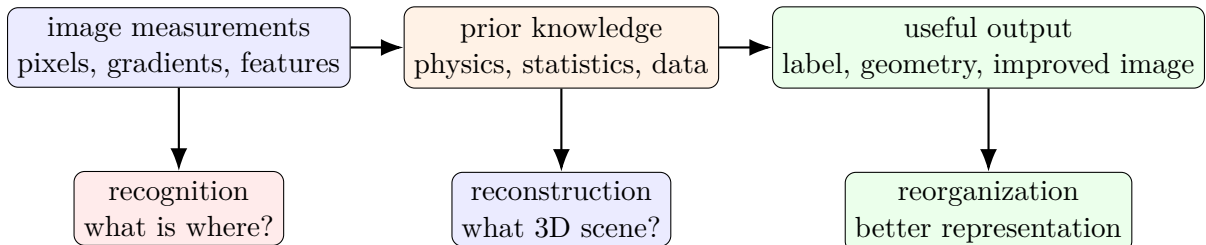


Figure 9: A compact summary of the introductory framing: measurements become useful outputs only after they are combined with assumptions, models, and data.

1.6 Topic Roadmap

Automatic image analysis is narrower than the whole field of computer vision. It uses image processing tools and needs some geometric and probabilistic reasoning, but the main line is automatic interpretation: how images are transformed into features, segments, object models, detections, categories, and evaluated decisions.

The foundations are image representation, filtering, features, transformations, and the basic language of image formation. These topics supply the bottom-up evidence used later. Segmentation, clustering, and feature extraction provide mid-level descriptions that make object-level reasoning possible. Learning-based methods, especially neural networks, show how useful representations and classifiers can be estimated from data. Object models, probabilistic estimation,

and higher-level recognition tasks connect these pieces; reconstruction appears where it helps explain image formation, geometry, or the limits of recognition.

Three questions recur throughout automatic image analysis:

1. What information is actually present in the image measurements?
2. What assumptions are needed to infer the hidden scene or decision?
3. How can we test whether the algorithm works beyond the example in front of us?

Automatic image analysis starts with pixels but does not end there. Its real task is to connect images to explanations: what is in the scene, how it is arranged, how it changes, and what action or decision should follow.

2 Neural Networks and Backpropagation

Neural networks are one way to replace manually designed decision rules by learned functions. For image analysis, the input may be an image, a patch, or a feature vector, and the output may be a class score, a probability, a segmentation label, or another task-specific prediction. The central idea is simple: compose many parameterized transformations and fit their parameters from examples.

Szeliski's treatment of deep learning, especially the part on deep neural networks and backpropagation, is a useful companion for this topic. The focus here is narrower: understand the computational structure of a feed-forward network, why backpropagation is the efficient way to compute the gradients needed for training, and how to tell whether a trained network will generalize.

2.1 From Perceptrons to Multilayer Networks

A perceptron is a linear classifier. Given an input vector x , weights w , and bias b , it predicts a binary label by thresholding an affine score:

$$f(x) = \begin{cases} 1, & w^\top x + b > 0, \\ -1, & \text{otherwise.} \end{cases} \quad (3)$$

The decision boundary is the hyperplane $w^\top x + b = 0$. This makes the perceptron easy to interpret, but also limited: if the classes cannot be separated by a linear boundary, the standard perceptron cannot solve the problem. The XOR problem is the usual minimal example.

During perceptron training, an incorrectly predicted example changes the weights in the direction of the input:

$$w_i^{t+1} = w_i^t + \eta (y - f(x)) x_i, \quad (4)$$

where η is the learning rate. The rule is useful historically because it already shows the pattern of learning from prediction error, but modern neural networks need a smoother and more general gradient-based view.

Logistic regression is the smooth relative of the perceptron and a useful reference point. For C classes and a D -dimensional input it computes one affine score per class, $z = Wx + b$ with $W \in \mathbb{R}^{C \times D}$ and $b \in \mathbb{R}^C$, and turns the scores into probabilities with the softmax function (introduced below). Its parameter count, $C(D + 1)$, therefore depends on both the number of classes and the input dimensionality (for two classes one score suffices, giving $D + 1$). Despite the nonlinear softmax, the predicted class is the largest affine score, and the boundary between two classes, where $z_i = z_j$, is a hyperplane: logistic regression is a *linear* classifier, exactly like linear regression used for classification.

A multilayer perceptron (MLP) overcomes the purely linear limitation by stacking several affine maps and nonlinear activation functions. For layer l ,

$$h^l = W^l x^l + b^l, \quad y^l = \phi(h^l), \quad x^{l+1} = y^l. \quad (5)$$

Without the nonlinearity ϕ , stacking layers would still collapse into one linear transformation. The nonlinearities make it possible to build complex decision boundaries from many simple units. The entries of W^l are the *weights*, one per connection, and b^l is the *bias*. Network diagrams often draw the bias as an extra neuron with constant output 1 whose outgoing weights are the bias values; both views compute the same affine map.

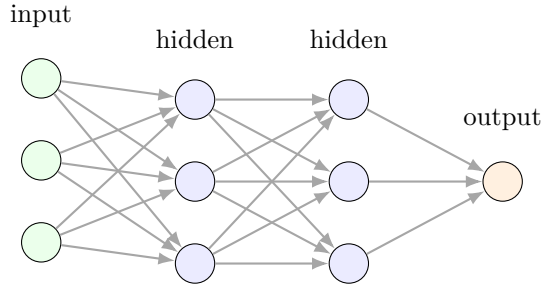


Figure 10: A feed-forward network composes affine transformations and nonlinearities. Training changes the weights on the connections.

Beyond being nonlinear, a useful transfer function is typically monotonic and smooth, so that it has a well-defined derivative everywhere; the classical logistic sigmoid and hyperbolic tangent are additionally bounded, which keeps activations in a fixed range. Common activation functions include the logistic sigmoid, hyperbolic tangent, and rectified linear unit:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad (6)$$

$$\tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}(z) = \max(0, z). \quad (7)$$

For ReLU, the derivative is 1 for positive inputs and 0 for negative inputs; at zero one chooses a convenient subgradient in implementations. This simple derivative is one reason ReLU-style activations are computationally attractive, and ReLU is the standard choice in convolutional networks.

Worked forward pass. Computing a small network by hand is the best check that the notation is understood. Take two inputs, one hidden layer with two ReLU units, and two ReLU output units, with the weights of Table 7. The input is $x = (2, 1)$ and the target is $y = (0, 1)$.

Unit	weight from 1st input	weight from 2nd input	bias
hidden h_1	1	-1	0
hidden h_2	-1	1	0.5
output o_1 (inputs h_1, h_2)	1	1	-1
output o_2 (inputs h_1, h_2)	2	-1	0

Table 7: Weights of a 2–2–2 ReLU network. Each row lists the weights of the connections entering one unit; the bias is the weight from the constant-1 neuron.

Layer by layer, first form the weighted sum and then apply ReLU:

$$h_1 = \max(0, 1 \cdot 2 - 1 \cdot 1 + 0) = 1, \quad h_2 = \max(0, -1 \cdot 2 + 1 \cdot 1 + 0.5) = \max(0, -0.5) = 0, \\ o_1 = \max(0, 1 \cdot 1 + 1 \cdot 0 - 1) = 0, \quad o_2 = \max(0, 2 \cdot 1 - 1 \cdot 0 + 0) = 2.$$

With the squared-error criterion $e = \frac{1}{2}\|o - y\|_2^2$ the error is $e = \frac{1}{2}((0 - 0)^2 + (2 - 1)^2) = \frac{1}{2}$. Two details are easy to get wrong under time pressure: ReLU clips negative sums to zero *before* they enter the next layer, and the factor $\frac{1}{2}$ belongs to the whole sum.

2.2 Losses, Optimization, and the Need for Gradients

Training starts with examples (x_i, y_i) and a network f_θ with parameters θ , meaning all weights and biases. A loss function measures how bad a prediction is. Over a dataset, the empirical objective is

$$L(\theta) = \sum_{i=1}^N \ell(f_\theta(x_i), y_i). \quad (8)$$

For regression, ℓ is typically the squared error $\frac{1}{2}\|f_\theta(x) - y\|^2$. For classification, the network outputs one score (logit) z_c per class, the softmax turns the scores into probabilities, and the loss is the cross-entropy, the negative log-probability of the correct class y :

$$p_c = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}}, \quad \ell_{\text{CE}} = -\log p_y. \quad (9)$$

The loss is what the optimizer sees, so it must match the task; accuracy itself is not differentiable and is only used to evaluate the result.

Even when the loss is convex, as for logistic regression, there is in general no closed-form solution for the best parameters, so the minimum has to be found iteratively. Gradient descent is therefore not a tool reserved for non-convex problems; convexity only guarantees that it will not get stuck in a worse local minimum. For deep networks the loss is non-convex, and (stochastic) gradient descent gives no guarantee of reaching the optimal solution.

Gradient descent steps against the gradient with learning rate η . The three classical variants differ only in how many examples enter one update:

$$\text{online: } \Delta w = -\eta \frac{\partial e_n}{\partial w} \quad \text{after every single example } n, \quad (10)$$

$$\text{mini-batch: } \Delta w = -\eta \sum_{n \in \mathcal{B}_t} \frac{\partial e_n}{\partial w} \quad \text{after a small batch } \mathcal{B}_t, \quad (11)$$

$$\text{offline (batch): } \Delta w = -\eta \sum_{n=1}^N \frac{\partial e_n}{\partial w} \quad \text{after a pass over all } N \text{ examples.} \quad (12)$$

Offline learning follows the exact gradient of the full objective but updates rarely; online learning updates often but with a noisy direction. Stochastic gradient descent in modern practice is the mini-batch variant. Two common additions modify the update. *Momentum* (inertia) keeps part of the previous step, so that consistent directions accelerate and oscillations cancel,

$$\Delta w(t+1) = -\eta \frac{\partial e}{\partial w} + \alpha \Delta w(t), \quad 0 \leq \alpha < 1; \quad (13)$$

weight decay adds a penalty $\frac{\lambda}{2}\|w\|^2$ to the loss, which contributes $-\eta\lambda w$ to every step and pulls unused weights toward zero.

Whatever the variant, every step needs the partial derivatives of the loss with respect to all parameters. The practical difficulty is not writing the update down; it is computing these derivatives for millions of parameters quickly and without duplicating work. Backpropagation solves exactly that problem.

Component	Role in the network	Training question
Weights and biases	Store the adjustable model parameters	How should each parameter change to reduce the loss?
Activations	Introduce nonlinear transformations between layers	Which units are active and how do they transmit gradients?
Loss function	Converts predictions and targets into one scalar	What does “better” mean for this task?
Optimizer	Turns gradients into parameter updates	How large and in which direction should the next step be?

Table 8: Training a neural network requires both a model architecture and a procedure for turning prediction errors into parameter updates.

2.3 Backpropagation

Backpropagation is an efficient application of the chain rule to a computational graph. The forward pass computes the network output and stores intermediate quantities. The backward pass starts from the derivative of the loss with respect to the output and propagates sensitivity information back through the same operations in reverse order.

A graph worked by hand. The mechanism is clearest on a tiny scalar graph, $f = (x + y) \cdot z$ with $x = -2$, $y = 5$, $z = -4$ (Figure 11). The forward pass computes and stores the intermediate value $q = x + y = 3$ and the output $f = qz = -12$. The backward pass starts with $\partial f / \partial f = 1$ and applies one local rule per node. The multiplication node sends to each input the upstream gradient times the *other* input: $\partial f / \partial z = q = 3$ and $\partial f / \partial q = z = -4$. The addition node has local derivative 1 for both inputs, so it passes the upstream gradient unchanged to each of them: $\partial f / \partial x = \partial f / \partial y = -4$.

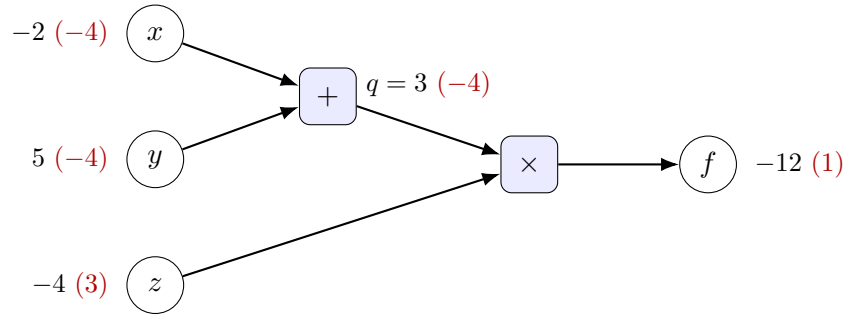


Figure 11: Backpropagation on $f = (x + y)z$. Black numbers are forward values, red numbers in parentheses are the gradients $\partial f / \partial (\cdot)$ from the backward pass. The multiplication node swaps its inputs; the addition node copies the upstream gradient to both inputs.

These local rules are worth remembering as patterns, summarized in Table 9. In particular, an addition node does *not* send its inputs backward; it distributes the upstream gradient.

The layer view. It is helpful to name the quantities at one layer before writing the gradient. Let h_k be the activation vector that enters layer k . The trainable parameters of this layer are a weight matrix Ω_k and a bias vector β_k . The affine part of the layer first forms the pre-activation vector

$$f_k = \beta_k + \Omega_k h_k, \quad (14)$$

Node	Local derivative	Backward behavior
Addition $a + b$	1 for each input	distributes: each input receives the upstream gradient unchanged
Multiplication ab	b for a , a for b	swaps: each input receives the upstream gradient times the other input's forward value
Maximum $\max(a, b)$	1 for the larger, 0 else	routes: only the larger input receives the gradient (ReLU is $\max(0, z)$)

Table 9: Backward rules of the elementary nodes. The multiplication rule is why forward values must be stored until the backward pass.

and a nonlinearity would then produce the next activation, for example $h_{k+1} = \phi(f_k)$. With this notation, the gradient with respect to the weight matrix has the structure

$$\frac{\partial \ell_i}{\partial \Omega_k} = \frac{\partial \ell_i}{\partial f_k} h_k^\top. \quad (15)$$

This is the multiplication rule at matrix scale: the effect of a weight depends both on the upstream error signal and on the activation feeding into that weight. If the previous activation is doubled, the same small weight change has twice the local effect.

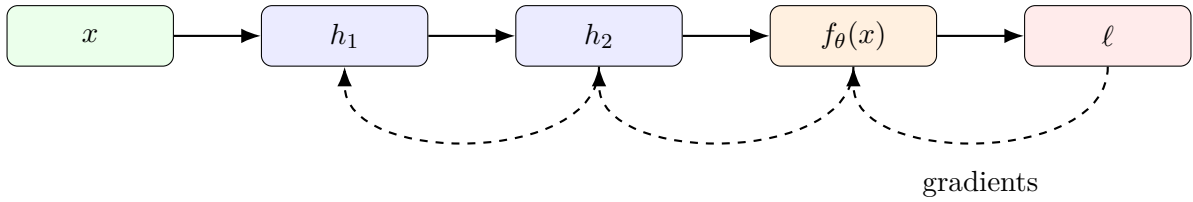


Figure 12: The forward pass evaluates the network from left to right. The backward pass propagates derivatives from the loss back through the stored intermediate quantities.

For ReLU hidden units, the backward recurrence includes the pointwise derivative of the activation:

$$\frac{\partial \ell_i}{\partial f_{k-1}} = \mathbb{I}[f_{k-1} > 0] \odot \left(\Omega_k^\top \frac{\partial \ell_i}{\partial f_k} \right), \quad (16)$$

where $\odot$ denotes elementwise multiplication. The indicator gates the gradient: inactive ReLU units do not pass local gradient information backward.

What backpropagation costs and what it does not do. The important point is not to memorize every indexed expression, but to know what must be stored and what is reused. One parameter update needs exactly *one* forward pass (to compute and store all intermediate values) and *one* backward pass (to reuse them); every derivative is computed once and shared by all parameters upstream of it. Because the forward values of every node are kept until the backward pass, the memory consumption grows with the combined output dimensionality of all nodes in the graph (the stored activations, times the batch size), not merely with the number of trainable parameters. Backpropagation is only a way to *compute* the gradient exactly and efficiently. It does not change which gradient is computed, so it neither reduces the noise of stochastic gradient descent (that comes from using mini-batches) nor guarantees that the optimizer reaches the optimal solution.

2.4 Automatic Differentiation and Practical Training

Modern deep-learning libraries implement backpropagation through automatic differentiation. A model is represented as an acyclic graph of operations. Each operation knows how to compute its output in the forward direction and how to propagate derivatives in the reverse direction, exactly as the nodes of Table 9. The user specifies the model, the loss, and the optimizer; the framework records the graph and applies the chain rule. Branches are allowed as long as the graph stays acyclic.

A typical training loop has the following structure:

1. initialize the network parameters;
2. choose a loss function and optimizer;
3. load a batch of examples;
4. run the forward pass and compute the loss;
5. run the backward pass to compute gradients;
6. update the parameters and repeat.

This is the conceptual core behind PyTorch-style code: zero old gradients, evaluate the model, call backward on the loss, and let the optimizer step. Poor initialization can make training slow or unstable. Too large a learning rate can overshoot; too small a learning rate can make progress painfully slow.

2.5 Generalization: Overfitting, Bias, Variance, and Regularization

A network that fits its training data perfectly may still be useless. The goal is low error on *new* data, and the training error cannot measure that: a flexible model can memorize the training set, noise included, which is called *overfitting*. The opposite failure, *underfitting*, occurs when the model is not complex enough to match the data and performs poorly even on the training set. Figure 13 shows the typical picture: as model complexity grows, the training error keeps falling, while the error on held-out data first falls and then rises again once the model starts fitting noise.

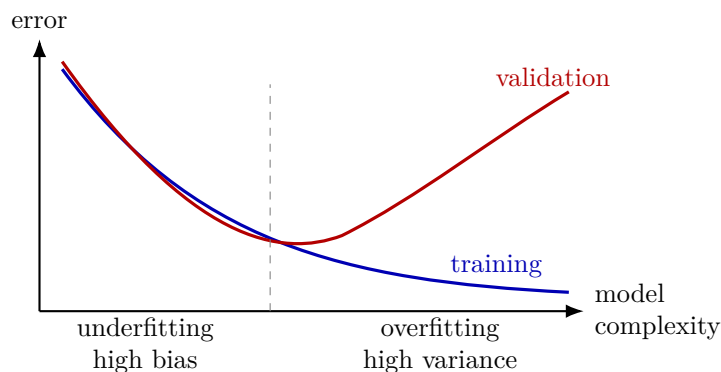


Figure 13: Training and validation error versus model complexity. The dashed line marks the complexity with the lowest validation error; to its left the model underfits, to its right it overfits.

The *bias–variance* vocabulary names the two failures. A model with high *bias* does not match the training data closely; it underfits. *Variance* describes how much the fitted model changes when it is trained on different portions of the data. It can be measured exactly that way: fit the model to several subsets and compare the results. A high-variance model follows the particular sample it saw and therefore overfits.

Training, validation, and test data. Because the training error is optimistically biased, the available data are split into three parts with different roles. The *training set* fits the parameters. The *validation set* is used for every decision about the model: hyperparameters such as learning rate, weight decay, or depth, and when to stop training. The degree of overfitting is read from the gap between training and validation error, never from the training data alone. The *test set* is touched once, at the end, for an unbiased estimate; once it influences a decision, it is no longer a test set. This separation is needed for every model, convex or not, because convexity concerns only how well the optimizer finds the minimum of the training loss, not how well that minimum generalizes. It also fixes the reading of small errors: a small training error only suggests that the model has sufficient complexity, while a small *validation* error suggests good performance.

Regularization. Regularization is any change that reduces overfitting by encoding a preference for simpler or more robust solutions. The standard tools are:

- *Weight decay* (L_2 penalty): add $\lambda\|W\|_2^2$ to the loss so that large weights are discouraged.
- *Dropout*: randomly set a fraction of activations to zero during training so that the network cannot rely on any single path; at test time all units are used.
- *Data augmentation*: enlarge the training set with transformed copies (crops, flips, small rotations, color jitter). This has a regularizing effect, because the model must produce the same answer for all variants; the transformations must respect the semantics of the task.
- *Early stopping*: monitor the validation loss after every epoch, stop when it has not improved for a fixed number p of epochs (patience), and keep the checkpoint with the best validation loss, not the last one.
- *Batch normalization*: normalize activations over the mini-batch and learn a scale and shift; mainly a stabilizer of optimization with a mild regularizing side effect.

More data and a smaller model act in the same direction. Table 10 turns these ideas into a diagnosis checklist.

Symptom	Meaning	Typical response
High training error	The model or optimization setup is too weak (high bias)	increase capacity, train longer, adjust learning rate
Low training error but high validation error	The model overfits the training set (high variance)	use regularization, more data, or early stopping
Unstable loss	Updates are too aggressive or scales are poor	lower learning rate, improve initialization, normalize inputs

Table 10: Most neural-network training failures can be diagnosed by comparing training behavior with validation behavior.

3 Visual Object Models

An object model is a computational answer to a deceptively simple question: what does an object look like, expressed in a form an algorithm can search for? A person has no trouble saying that a coffee mug has a rounded body and a handle, but a detector needs something concrete: a parametric shape, a set of edge displacements, a collection of characteristic image patches, or a learned feature map. The shape of that representation decides which transformations the

detector can tolerate (translation, scale, rotation, partial occlusion) and how the search through the image is organized.

This section develops object models through a single recurring idea that ties its parts together:

Local image evidence accumulates into global object hypotheses by voting.

Each fragment of the image (an edge, a gradient, a patch) is weak and ambiguous on its own, but it is consistent with only a limited set of global explanations. If every fragment casts a vote for the explanations it supports, the correct hypothesis collects many votes and stands out as a peak in a parameter space, while accidental coincidences stay near the noise floor. We follow this idea through increasingly flexible models: edges first vote for *lines* (Hough transform), then for the center of a *known shape* (generalized Hough transform), and finally learned appearance *parts* vote for the center of an *object category* (implicit shape model). Saliency and descriptors sit in between, deciding *which* local evidence is worth trusting in the first place.

3.1 Detecting Lines: The Hough Transform

The simplest non-trivial object model is a straight line, possibly broken by noise or occlusion into many short edge fragments. The difficulty is that the fragments are scattered across the image, and we do not know in advance how many lines there are or where they lie. Testing every possible line against every group of pixels would be hopeless. The Hough transform sidesteps this by changing coordinates: instead of grouping pixels in the image, it lets every pixel vote in the space of all possible *lines*, and then simply looks for the lines that received many votes.

To make “the space of all lines” concrete we need a way to label each line by a few numbers. The familiar slope–intercept form $y = mx + b$ is a poor choice: a vertical line has infinite slope, so m and b cannot represent it. The *Hesse normal form* sidesteps this by describing a line through the one point it can never help but single out — the point on the line closest to the origin.

Every line has a unique closest point to the origin: the foot of the perpendicular dropped from the origin onto the line. We can reach that foot by starting at the origin and walking a fixed distance $r \geq 0$ in a single direction. Call θ the angle of that direction, so the unit vector pointing from the origin straight at the line is

$$\hat{\mathbf{n}} = (\cos \theta, \sin \theta), \quad (17)$$

so the foot of the perpendicular is the point

$$P_0 = r \hat{\mathbf{n}} = (r \cos \theta, r \sin \theta). \quad (18)$$

The vector from the origin to P_0 is r times $\hat{\mathbf{n}}$, so it is *normal* (perpendicular) to the line, because the shortest path from a point to a line always meets it at a right angle. The pair (r, θ) then pins the line down completely: θ says which way the line faces and r says how far out it lies.

To turn this picture into an equation, take any point $P = (x, y)$ on the line and look at the vector $P - P_0$ that runs from the foot to it. Because both P and P_0 lie on the line, this vector points *along* the line; and the line is perpendicular to the origin-to-foot vector P_0 . Orthogonality of $P - P_0$ and P_0 means their dot product vanishes,

$$(P - P_0) \cdot P_0 = 0 \quad \Longleftrightarrow \quad P \cdot P_0 = P_0 \cdot P_0. \quad (19)$$

Both sides are now immediate. On the left, $P \cdot P_0 = x(r \cos \theta) + y(r \sin \theta) = r(x \cos \theta + y \sin \theta)$; on the right, $P_0 \cdot P_0 = r^2(\cos^2 \theta + \sin^2 \theta) = r^2$, using $\cos^2 \theta + \sin^2 \theta = 1$. Equating the two gives $r(x \cos \theta + y \sin \theta) = r^2$, and cancelling one factor of r (every line we care about has $r > 0$) leaves the Hesse normal form of the line,

$$x \cos \theta + y \sin \theta = r. \quad (20)$$

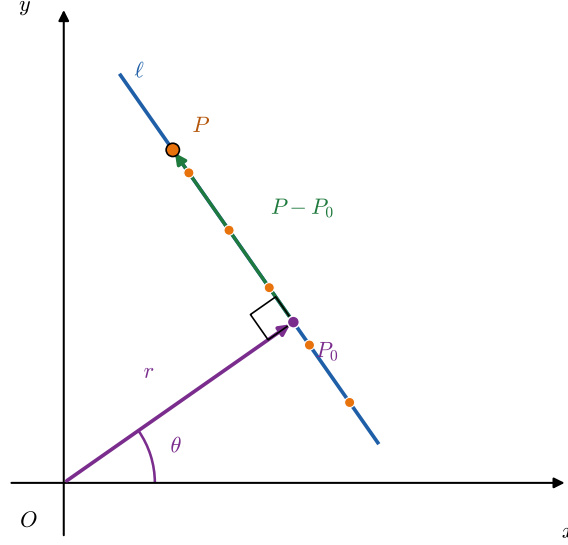


Figure 14: The Hesse normal form of a line. The foot of the perpendicular from the origin O is $P_0 = (r \cos \theta, r \sin \theta)$, reached by walking a distance r at angle θ . For any point P on the line ℓ , the vector $P - P_0$ runs along the line and is therefore orthogonal to P_0 ; the condition $(P - P_0) \cdot P_0 = 0$ is exactly $x \cos \theta + y \sin \theta = r$, which every collinear edge point (orange) satisfies with the same (r, θ) .

Figure 14 shows the whole construction. What looked like an arbitrary combination of a cosine and a sine is really one clean geometric statement: from every point P on the line, the step $P - P_0$ to the foot leaves at a right angle to the origin-to-foot vector P_0 .

Here x and y are pixel coordinates, $r \geq 0$ is measured in pixels, and θ ranges over $[0, \pi)$. A quick sanity check fixes the meaning of the two numbers. A horizontal line at height $y = c$ has a vertical perpendicular, so $\theta = 90^\circ$ and Equation (20) reduces to $r = c$; a vertical line $x = d$ has $\theta = 0^\circ$ and $r = d$; a line passing through the origin has $r = 0$ at every one of its points, whatever its angle. No line, not even a vertical one, makes the parameters blow up. That is exactly the robustness the slope–intercept form lacks.

The decisive property of Equation (20) is an *invariance*: every pixel that lies on one particular line produces the *same* pair (r, θ) , because r and θ describe the line, not the individual point. A whole line in the image therefore collapses to a single point in the (r, θ) *parameter space*. Now read the same equation the other way around. Fix a single image point (x_0, y_0) and let θ vary; the equation turns into a function of θ alone,

$$r(\theta) = x_0 \cos \theta + y_0 \sin \theta, \quad (21)$$

a fixed weighted sum of a cosine and a sine. Any such combination collapses to a single shifted sine of the same frequency, and it is worth seeing exactly why, because the shift is where the phase φ comes from. Write down the shifted sine we are aiming for and expand it with the angle-addition identity $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$:

$$R \sin(\theta + \varphi) = \underbrace{R \cos \varphi}_{\text{weight of } \sin \theta} \sin \theta + \underbrace{R \sin \varphi}_{\text{weight of } \cos \theta} \cos \theta. \quad (22)$$

For this to equal $x_0 \cos \theta + y_0 \sin \theta$ for *every* θ , the two expressions must have the same weight on $\cos \theta$ and the same weight on $\sin \theta$ separately. Matching them term by term gives

$$x_0 = R \sin \varphi, \quad y_0 = R \cos \varphi. \quad (23)$$

These two relations determine R and φ from the point alone. Squaring and adding them cancels φ (via $\sin^2 + \cos^2 = 1$) and leaves the amplitude,

$$R = \sqrt{x_0^2 + y_0^2}, \quad (24)$$

which is simply the point's distance from the origin; dividing them instead cancels R and fixes the phase,

$$\varphi = \text{atan2}(x_0, y_0). \quad (25)$$

Here atan2 is the two-argument arctangent, which recovers the angle in the correct quadrant from the signed pair (x_0, y_0) . So φ is not a new free parameter that appears from nowhere: it is the phase *forced* on the sinusoid by where the point sits, namely the angle of (x_0, y_0) measured from the y -axis (that is what Equation (23) says, since $(x_0, y_0) = (R \sin \varphi, R \cos \varphi)$). Reassembling the pieces,

$$r(\theta) = x_0 \cos \theta + y_0 \sin \theta = R \sin(\theta + \varphi), \quad (26)$$

which is a sinusoid in the (r, θ) plane whose amplitude R is just the point's distance from the origin and whose phase φ encodes its direction. The sinusoid enumerates *all* the lines that pass through that one point. Two pixels that happen to lie on a common line have sinusoids that cross at the (r, θ) of that shared line; many collinear pixels produce many sinusoids crossing at a single point. That crossing is the peak we are looking for. Figure 19 shows both readings side by side: collinear points in the image (one point, one sinusoid) versus the single bright accumulator cell where the sinusoids meet.

So far the “points” that vote have been abstract, but on a real image they are anything but. We do not start with a tidy list of points; we start with a grid of brightness values, and the first job is to decide which pixels are even worth a vote. That is what *edge detection* does: it flags the pixels that sit on a strong local change in brightness — the outlines where a dark region meets a light one — and leaves the smooth interior, which carries no boundary information, silent. These high-contrast *edge pixels* are the evidence the transform actually works with. A straight edge in the scene appears as a run of collinear edge pixels, and it is exactly their sinusoids that all pass through the one (r, θ) of that edge. A peak in parameter space is therefore not a geometric coincidence but a direct tally of how many real edge pixels line up: the more pixels that genuinely share a line, the taller the crossing they build, and the further it stands above the scatter left by unrelated pixels.

This is also where the name *Radon transform* comes from. We cannot solve for (r, θ) directly, because a single point lies on infinitely many lines. So we discretize the parameter space into a grid of cells, an *accumulator* $H_L(r, \theta)$ initialized to zero, and let each edge pixel add one vote to every cell on its sinusoid: for the pixel, sweep θ over the grid, compute the matching r from Equation (20), and increment $H_L(r, \theta)$. After all pixels have voted, a real line shows up as a pronounced maximum, while cells that received only scattered, accidental votes stay low. Detecting lines has become peak-finding in a 2D array.

It helps to picture H_L literally as an image and to set it next to the picture it came from, which is what Figure 15 does: the edge map on the left, the accumulator on the right. The horizontal axis is the angle θ and the vertical axis is the distance r — here allowed to take either sign, so that θ needs to run only over a half-turn instead of a full one. The plane is chopped into a grid of *bins*, and the number of bins along each axis sets the resolution of the search: too coarse and neighbouring lines merge into one cell, too fine and the votes for a single line scatter across several. The grid starts completely black, at zero. Each edge pixel then paints its sinusoid of votes across it, brightening exactly one bin in every θ column, so on its own a pixel leaves only a faint curve. Where many collinear pixels overlap, their sinusoids pile into the *same* bin, which flares into a concentrated bright hotspot, while the accidental votes of unrelated pixels stay dim and diffuse. Detecting lines is then nothing more than thresholding this image for its brightest cells and reading off their (r, θ) .

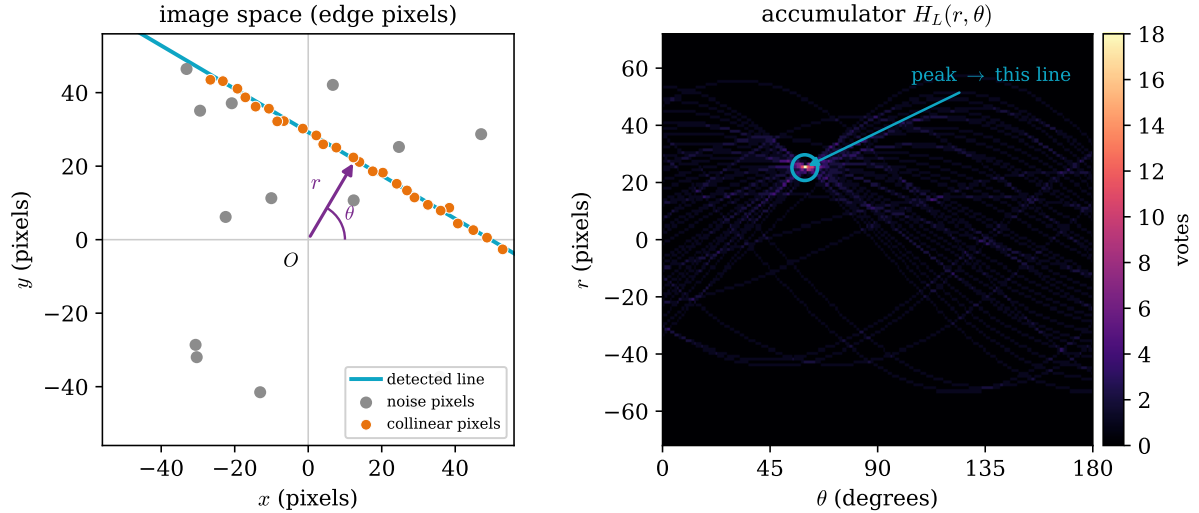


Figure 15: Image space and parameter space side by side, for a synthetic edge map of 26 collinear pixels (orange) plus 14 scattered ones (grey). *Left*: the edge pixels, with the detected line and its perpendicular parameters r and θ measured from the origin O . *Right*: the accumulator $H_L(r, \theta)$ drawn as an image. Every edge pixel casts one vote per θ column, tracing a sinusoid across the grid; the collinear pixels’ sinusoids all cross in the same bin, which lights up as the bright peak (circled) at $(\theta, r) \approx (60^\circ, 25 \text{ px})$, whereas the noise pixels leave only faint, uncrossed curves. That peak *is* the line on the left: line detection reduces to thresholding this grid for its hottest cells and reading off their (r, θ) . Brightness is the number of votes a bin has received.

Using gradients. Making every edge pixel sweep over all directions θ is wasteful, because it throws away information the edge detector already handed us for free. Finding an edge means measuring, at each pixel, the direction in which the brightness changes fastest: the direction of *steepest contrast*, pointing straight across the boundary from the dark side to the light side. That direction is the image *gradient*. Now the geometric twist that makes it so useful — and it is easy to get backwards. The brightness changes fastest as you step *across* an edge and barely at all as you walk *along* it, so the gradient points *perpendicular* to the edge, not along it. The direction of maximum contrast tells us which way the boundary faces; the edge itself runs at a right angle to that. But “perpendicular to the edge” is precisely the *normal* direction of the line through the pixel — the same normal $\hat{\mathbf{n}}$ whose angle we named θ in the Hesse form. The contrast direction therefore hands us θ directly, and with it the single line that can pass through that pixel, so the pixel casts one sharp, well-aimed vote instead of a whole sinusoid of guesses. To make this concrete, write the gradient of the image brightness as the pair $(G_x(x, y), G_y(x, y))$, the rates of change in the x and y directions (for example the outputs of a Sobel filter). The gradient direction

$$\theta_G(x, y) = \text{atan2}(G_y(x, y), G_x(x, y)) \quad (27)$$

already fixes the angle θ of the line through that pixel, so there is no need to try every θ : we compute the single matching r from Equation (20) and update one cell instead of a whole sinusoid. (The function atan2 is the two-argument arctangent; unlike $\arctan(G_y/G_x)$ it returns the angle in the correct quadrant and does not divide by zero when $G_x = 0$.) Two conventions matter when reading exercises: gradients point from dark to bright, and angles are measured counter-clockwise from the x -axis. How G_x and G_y are computed with derivative filters is summarized in Appendix B. Finally, a sharp, high-contrast edge is stronger evidence for a line than a faint one, so instead of adding a flat $+1$ we weight each vote by the gradient magnitude,

$$H_L(r, \theta) \leftarrow H_L(r, \theta) + \sqrt{G_x(x, y)^2 + G_y(x, y)^2}, \quad (28)$$

where $\sqrt{G_x^2 + G_y^2}$ is the length of the gradient vector, that is, how steeply the brightness changes at the pixel. A line in the image then produces a pronounced maximum in H_L , found by the same simple peak detection as before.

Points versus edges. The contrast between voting with bare points and voting with oriented edges is worth keeping in mind, because it recurs in every later method. A bare point is consistent with infinitely many lines and must vote for all of them (its full sinusoid), which inflates the chance that unrelated points accidentally agree and forces a higher peak threshold. An oriented edge, by contrast, carries a direction, pins down θ through Equation (27), and votes for essentially one line. The lesson generalizes: the more informative each piece of local evidence is, the cleaner the peaks and the fewer the false alarms. The two modes carry two names. Letting a point sweep over all θ and trace its full sinusoid is the *Radon* view: a point in image space becomes a sinusoid in parameter space. The gradient-based *Hough* line detector maps an oriented edge pixel to a single point in parameter space.

Working with the parameterization. Three kinds of exercise follow directly from Equation (20). First, to test whether a line (θ, r) passes through a point, substitute the point and compare. For the point $(x, y) = (3, 7)$:

(θ, r)	$x \cos \theta + y \sin \theta$	on the line?
$(0.5\pi, 3)$	$3 \cdot 0 + 7 \cdot 1 = 7$	no ($7 \neq 3$)
$(0, 7)$	$3 \cdot 1 + 7 \cdot 0 = 3$	no ($3 \neq 7$)
$(0, 3)$	3	yes
$(0.75\pi, -7)$	$3 \cdot (-\frac{\sqrt{2}}{2}) + 7 \cdot \frac{\sqrt{2}}{2} = 2\sqrt{2} \approx 2.83$	no

The pattern to remember: $\theta = 0$ describes the vertical line $x = r$, $\theta = \pi/2$ the horizontal line $y = r$.

Second, to count the votes a pixel casts, count the parameter cells it increments. Suppose θ is discretized in 1° steps over $0^\circ, \dots, 359^\circ$ and r over $-100, \dots, 100$. Without gradient information a point votes once per θ column, 360 votes (one r per θ , never all 201 values of r). With its gradient (G_x, G_y) the orientation θ is fixed by Equation (27), r follows from the Hesse form, and the pixel casts exactly *one* vote.

Third, to recognize the shape behind a voting space, count the peaks. Every straight segment is one line and produces one peak; two parallel sides share θ and differ only in r , so they give two peaks in the same column. A triangle therefore gives three peaks, a square or parallelogram four (in two pairs), and a circle none at all: each of its tangent lines touches only one point, so the votes spread evenly over a band $|r| \leq R$ whose edges are merely brighter where many tangents nearly coincide. Figure 16 shows these four voting spaces; a space with three maxima can only come from the triangle. By the same logic, two isolated pixels produce two sinusoids that cross in exactly one cell, the unique line through both of them. Circles are detected with a different parameterization, $(x - x_c)^2 + (y - y_c)^2 = R^2$, whose voting space is three-dimensional (x_c, y_c, R) .

3.2 Detecting Arbitrary Shapes: The Generalized Hough Transform

Lines are special because they have a tidy two-parameter equation. Most objects do not: a wrench, a leaf, or a logo is an arbitrary 2D curve with no closed-form parameterization. Since we cannot write such a shape down as a formula, we instead *show* it to the algorithm: we supply a *template* — a single example, one image of the object or equivalently the 2D curve of its boundary — that serves as the known geometric model of exactly the shape we want to find. The *generalized Hough transform* (GHT) keeps the voting idea but, in place of a parametric equation, compiles this template into a stored table that records the shape's geometry explicitly. The detector still

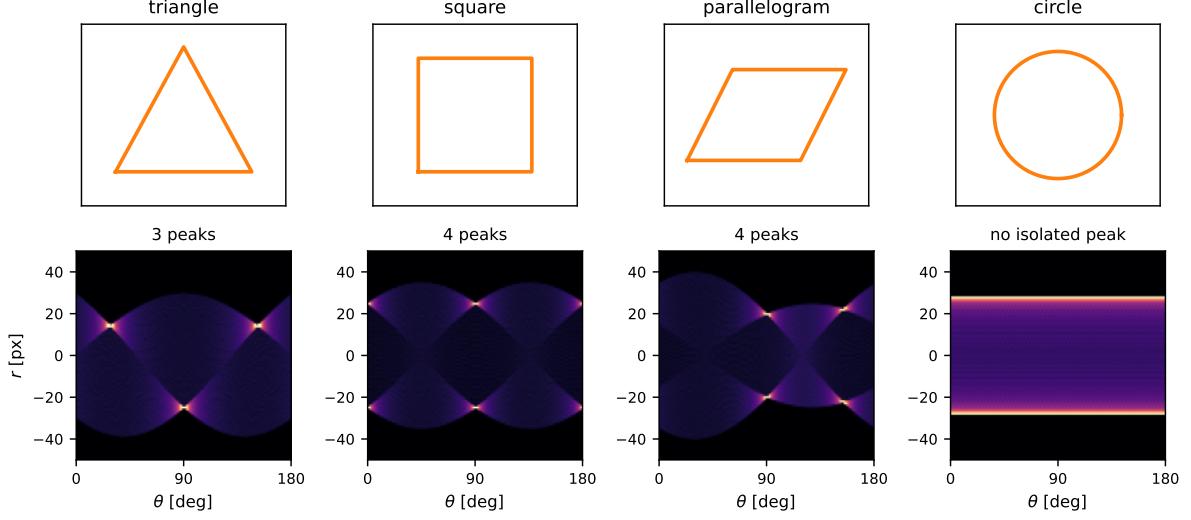


Figure 16: Line voting spaces of four outlines (every contour point votes for all $\theta \in [0^\circ, 180^\circ)$, signed r). Each straight side is one peak: three for the triangle, four for the square and the parallelogram, whose parallel sides appear as peak pairs at the same θ . A side at $\theta = 0^\circ$ reappears at the right border as $\theta = 180^\circ$ with opposite sign of r , which is the same line. The circle has no isolated peak, only a uniform band between $r = -R$ and $r = R$.

asks the same question every edge can answer with a vote, only now about a shape rather than a line: *if you are part of this shape, where is its center?*

Everything then splits into two phases built around that template, laid out in Figure 17. *Training* reads the template once and turns it into the table; *detection* runs the votes on each new test image. We take the two in turn.

To set this up we first pick one *reference point* on the template, typically (but not necessarily) its center of mass. This reference point plays the role that (r, θ) played for lines: its position in a test image is what we are voting for. During training we walk along the template's contour, and for each edge pixel at position (x, y) we record the *displacement vector* (d_x, d_y) that points from that edge to the reference point, so that

$$(x_{\text{ref}}, y_{\text{ref}}) = (x + d_x, y + d_y). \quad (29)$$

The logic at detection time is the mirror image of Equation (29): if an observed edge in the test image is the same part of the shape, then the center must lie at the edge's own position plus one of the stored displacements, so the edge votes for those locations. Edges that genuinely belong to one instance of the shape all point back to the same spot, which lights up as a peak; unrelated edges scatter their votes harmlessly.

What keeps this efficient is the *R-table*, an indexed list of the displacement vectors. Just as the gradient orientation accelerated line detection, it organizes the table here. We quantize the possible gradient directions into A equal bins (rows). Row T covers gradient orientations near $\theta \approx 2\pi T/A$ for $T \in \{0, 1, \dots, A-1\}$, and it stores every displacement vector that was observed at an edge with roughly that orientation:

$$\text{row } T : (d_x^{T,1}, d_y^{T,1}), (d_x^{T,2}, d_y^{T,2}), \dots, (d_x^{T,N_T}, d_y^{T,N_T}). \quad (30)$$

Concretely, with $A = 8$ bins each row spans 45° ; an edge whose gradient points at 100° falls in the row centered near 90° , and we read off exactly that row's vectors. The superscripts in Equation (30) just enumerate the N_T different vectors stored in row T — there can be several, which is the point of the next paragraph. At test time, then, the procedure is: read each edge's

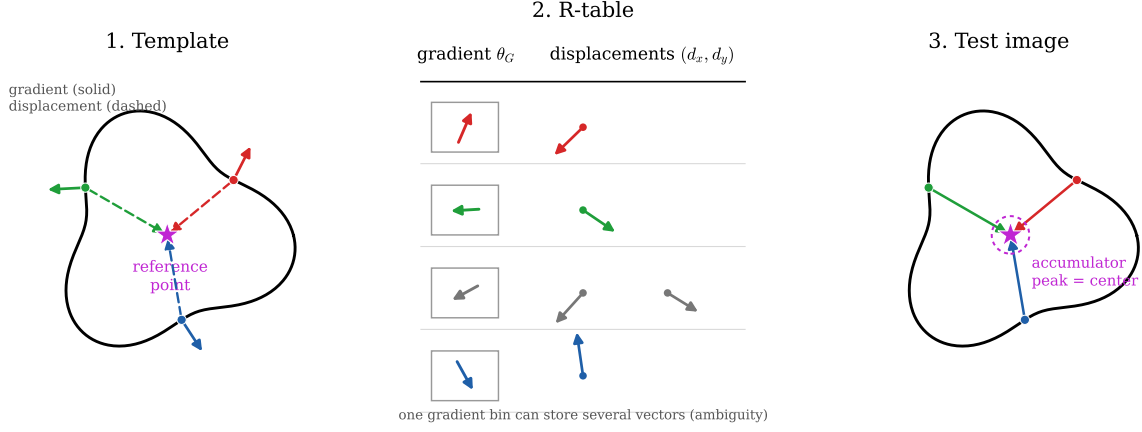


Figure 17: The generalized Hough transform, from template to detection. (1) *Template*: on the known model shape we fix a reference point (the center, magenta star); at each edge we read its gradient direction (solid arrows) and record the displacement vector pointing to the center (dashed arrows). (2) *R-table*: those displacements are filed by gradient orientation, so a gradient direction indexes straight to the vectors seen at edges facing that way; because one orientation can occur at several places on the shape, a single bin may hold more than one vector. (3) *Test image*: each observed edge looks up its gradient in the R-table and votes for a center at its position plus the stored displacement; edges of a genuine instance all point to the same spot, which wins the accumulator peak. Colours track the same edge through all three panels.

gradient orientation, jump to the matching row, and cast a vote at $(x + d_x, y + d_y)$ for every displacement (d_x, d_y) in that row.

Why does one row hold several vectors? Because the local gradient is *ambiguous*: the same gradient direction occurs at several different places on a shape (think of the two opposite sides of a circle, whose outward normals are antiparallel, or the many points along a smooth curve that happen to face the same way). A given orientation is therefore consistent with several possible centers, so a single observed edge generally increments *several* accumulator cells, one per displacement in its row. This is not a defect to be engineered away; it is precisely why voting is needed. Each edge is individually uncertain about where the center is, but only the true center collects support from many edges at once. Figure 20 traces this from the model contour and its displacement vectors through the R-table to the accumulator peak.

Real instances also differ from the stored template in size and orientation. To handle this, the accumulator gains two more axes and becomes four-dimensional, $H(x, y, s, \theta)$, where (x, y) is the candidate center, s a scale factor, and θ a rotation angle. The two phases are then:

Training. Extract the edge pixels of the template; for each, compute its gradient orientation and store the displacement vector to the reference point in the corresponding R-table row (Equation (30)).

Prediction. For each scale s , multiply the stored displacement vectors by s ; for each rotation θ , rotate the vectors by θ and shift the R-table rows around by the same amount (a circular shift, since rotating the shape rotates every edge’s gradient by θ); then, for each edge pixel in the test image, look up its row, cast a vote from each of its displacement vectors, and increment $H(x, y, s, \theta)$. Finally, find the peaks in the four-dimensional array.

The price of this generality is the four-dimensional search; the reward is a detector for an *arbitrary* known shape rather than a single parametric family of lines.

Transforming an R-table. The prediction loop builds a modified R-table for every candidate (s, φ) , and exercises ask for exactly this step. Three rules cover every case. A *scale* s multiplies every displacement vector and leaves the rows alone, since scaling does not change gradient directions. A counter-clockwise *rotation* by φ rotates every displacement vector,

$$\begin{pmatrix} d'_x \\ d'_y \end{pmatrix} = s \begin{pmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{pmatrix} \begin{pmatrix} d_x \\ d_y \end{pmatrix}, \quad (31)$$

and rotates every gradient by φ , so the entries of the row for orientation θ move to the row for $\theta + \varphi$: a circular shift by $\varphi/\Delta\theta$ rows, where $\Delta\theta$ is the bin width. If the table only covers orientations in $[0^\circ, 180^\circ)$, orientations are taken modulo 180° . A *translation* changes nothing, because the displacements are relative to the reference point, which moves along with the shape. Three worked cases show how the rules combine:

- Rows of width 45° over $[0^\circ, 180^\circ)$, $s = 2$, $\varphi = 180^\circ$. A rotation by 180° maps (d_x, d_y) to $(-d_x, -d_y)$, and $\theta + 180^\circ \equiv \theta \pmod{180^\circ}$ keeps every entry in its row. With the scale, $(1, 2)$ becomes $(-2, -4)$, $(9, 10)$ becomes $(-18, -20)$, and the empty row stays empty.
- Rows of width 90° over $[0^\circ, 360^\circ)$, $s = 1$, $\varphi = 180^\circ$, translation $t = (10, 10)$. Every vector is negated, and every row moves two places on: the entries of $[0, 90)$ go to $[180, 270)$, those of $[180, 270)$ to $[0, 90)$, those of $[270, 360)$ to $[90, 180)$. The translation is ignored.
- $s = 2$, $\varphi = 45^\circ$, rows of width 45° . With $\cos 45^\circ = \sin 45^\circ = 1/\sqrt{2}$, Equation (31) becomes $(d'_x, d'_y) = \sqrt{2}(d_x - d_y, d_x + d_y)$, for example $(2, 0) \mapsto (2\sqrt{2}, 2\sqrt{2})$, and each entry moves one row on.

The same rules explain which invariances a single R-table has (Table 11).

Property	Holds?	Reason
Invariant to translation	yes	displacements are relative to the reference point
Invariant to scale	no	displacements must be multiplied by s
Invariant to rotation	no	vectors must be rotated and rows shifted
Robust to noise and partial occlusion	yes	each edge votes on its own; a missing part lowers the peak, noise adds scattered votes
Needs gradient orientation at (x, y)	yes	the orientation selects the row to vote with

Table 11: Properties of a single R-table. Scale and rotation are handled by the extra accumulator dimensions, not by the table itself.

The dimensionality of the voting space follows from what is unknown: a line detector votes in 2D (r, θ) , a circle detector in 3D (x_c, y_c, R) , and the generalized Hough transform in 4D (x, y, s, θ) . An alternative to explicit R-tables correlates complex gradient images of template and image, which can be computed with the Fourier transform (Appendix C).

3.3 Where to Look: Salient Regions

The generalized Hough transform still assumes we possess an exact template of the object. That assumption breaks for object *categories*: no two cars, faces, or motorbikes share one outline, so there is no single contour to store in an R-table. Before we can let parts vote for an object, we have to decide which image locations even deserve to be treated as parts. This is the problem of *saliency*: detecting distinctive regions whose local appearance stands out from their surroundings, so that the same regions can be found again on a new, previously unseen instance.

The motivation is partly biological. Saliency is a subjective perceptual quality that makes certain items capture an observer’s attention before others. A standard computational account mirrors early vision: simple features (color, edge orientation, luminance, motion) are measured in parallel across the whole field and at several scales, producing a stack of *feature maps*. Each map is processed to emphasize locations that differ from their neighbors, the maps are combined into a single saliency map, and attention — and, in a living visual system, the eyes and head — is drawn to the most salient locations. The idea we borrow for object models is this notion of “standing out from the surroundings,” made mathematically precise and, importantly, invariant to scale and rotation so that detections repeat across instances.

The Kadir/Brady detector makes “standing out” concrete by equating saliency with *rareness*, measured through Shannon entropy. The reason rareness needs a careful definition is that it is easy to get trivially wrong: if our description of a patch is so specific that every location looks unique, then everything is equally rare; if it is so generic that everything looks alike, nothing is rare. Entropy of the local intensity distribution sits between these extremes and measures the *complexity* of a patch. For a window of pixels, let $p(k)$ be the fraction of pixels whose grey value equals k , for $k = 0, \dots, 255$; this is just the normalized local histogram. The entropy is

$$H = - \sum_{k=0}^{255} p(k) \log_2 p(k), \quad \text{with the convention } 0 \log_2 0 = 0. \quad (32)$$

Each term $-p(k) \log_2 p(k)$ is the contribution of grey level k ; summing them gives the average number of *bits* needed to encode a pixel drawn from the window, which is also a measure of how uncertain we are about that pixel’s value. A couple of concrete windows fix the intuition. If every pixel has the same grey value, one $p(k)$ equals 1 and the rest are 0, so $H = 0$: a flat, predictable patch carries no information. If the window contains two grey values in equal numbers, two terms equal $-\frac{1}{2} \log_2 \frac{1}{2} = \frac{1}{2}$ and $H = 1$ bit. If all 256 levels are equally likely, $p(k) = 1/256$ for each and $H = \log_2 256 = 8$ bits, the maximum. So a *flatter* local histogram means *higher* entropy, and a textured, unpredictable patch is rare in this spatial sense, while a smooth or constant patch is not.

Because Equation (32) depends only on the *list* of histogram values, several properties follow immediately:

- A patch whose histogram is uniform over all grey levels, such as pure noise, has the highest possible entropy.
- Relabeling grey values, for example swapping two of them, only permutes the histogram bins and does not change the entropy.
- Permuting the pixels leaves the histogram, and hence the entropy, unchanged. A checkerboard and a single straight edge with the same 50/50 black-white split both have $H = 1$ bit.
- A homogeneous patch ($H = 0$) has lower entropy than a patch containing an edge ($H > 0$).

The third property also shows the weakness of the measure: the intensity distribution contains no spatial information. Ranking patches by entropy is therefore a matter of counting grey levels and their frequencies. A constant patch ($H = 0$), a patch half black and half white ($H = 1$), a patch with four equally frequent grey levels ($H = 2$), and a noise patch (H close to 8) are in ascending order of entropy, whatever the arrangement of their pixels.

High spatial entropy alone is still not enough, because pure noise also has a flat histogram yet is not distinctive: its statistics barely change as the analysis window grows, so it looks the same at every scale. We therefore add a second requirement, *self-dissimilarity in scale*: a salient region should look different at its own scale than at nearby scales. To express this we analyze each location over a range of window sizes (a Gaussian scale space, or an image pyramid that

subsamples in logarithmic steps), and treat entropy as a function of the scale s . Writing $p(I, s, x)$ for the probability density of intensity value I inside a window of scale s centered at position x , the scale-dependent entropy and the scales at which it peaks are

$$H_D(s, x) = - \int p(I, s, x) \log_2 p(I, s, x) dI, \quad s_p = \left\{ s : \frac{\partial H_D}{\partial s} = 0, \frac{\partial^2 H_D}{\partial s^2} < 0 \right\}. \quad (33)$$

Equation (33) is just Equation (32) written for a continuous intensity I and made a function of scale; the condition on s_p is the textbook test for a local maximum (first derivative zero, second derivative negative), so s_p collects the scales where the entropy curve has a *peak*. Each such peak is then reweighted by how strongly the intensity distribution itself changes with scale,

$$W_D(s, x) = s \int \left| \frac{\partial}{\partial s} p(I, s, x) \right| dI, \quad y(s_p, x) = H_D(s_p, x) W_D(s_p, x). \quad (34)$$

The weight W_D is large when the distribution looks markedly different at scale s than at neighboring scales (the $|\partial p / \partial s|$ term), with the leading s making the measure comparable across scales; the final saliency $y(s_p, x)$ is the product, so a location scores highly only when it is both complex (high H_D) and tied to a distinctive scale (high W_D). This is exactly what suppresses featureless noise, whose entropy is high but flat across scale. Figure 21 shows the entropy-versus-scale curve with its peak, alongside the flat-versus-peaked histogram intuition.

The two multi-scale representations mentioned above are not the same thing. An *image pyramid* subsamples the image in logarithmic steps of resolution, $R_0, R_{-1}, R_{-2}, \dots$, so each level is smaller. A *linear Gaussian scale space* keeps the full resolution and instead smooths the original signal $L(x)$ with Gaussian kernels of growing width t ,

$$L(x, t) = L(x) * G(x, t), \quad G(x, t) = (\pi t)^{-n/2} \exp\left(-\frac{|x|^2}{t}\right), \quad (35)$$

where n is the dimension of x ($n = 2$ for images). A Gaussian scale space is therefore a family of images, all derived from the original and progressively more blurred as t increases; it is not a pyramid with randomly drawn scales, nor an assumption about how object sizes are distributed.

The scale weight also decides exercises that ask for the *most salient* patch. Saliency in the sense of Kadir and Brady is $y = H_D \cdot W_D$ at a peak scale, not the entropy alone. A noise patch has the highest entropy of all, yet its statistics are the same at every scale, so W_D is small and the patch is not salient. When a task only supplies entropy values, the ranking comes from those values, with patches that are known to be scale-less noise discounted. Entropy of the intensity distribution alone is not sufficient as a saliency measure because that distribution contains no spatial information.

In practice the algorithm visits each pixel, computes the local entropy (Equation (32)) over a range of scales, keeps the scales where the entropy peaks (there may be none), and weights those entropies by the inter-scale change of the distribution. Run on a dataset such as side views of motorbikes, and averaged over many images, the detected salient points cluster on repeatable, characteristic object parts — wheels, the engine block, the seat — which is precisely the raw material the next models need.

This is the *classical* stance on saliency: hand-crafted features grounded in information theory, computed without learning, and largely task-independent (data enter only at evaluation time). The *modern* alternative is to learn saliency from data with deep networks, building on the multilayer perceptrons and convolutional networks of Section 2; a convolutional network trades hand-designed rareness for features learned from many labeled examples, at the cost of needing a great deal of such data. We stay with the classical view here because it makes explicit *why* a location is worth voting from.

3.4 What to Record: Local Features and Shape Descriptors

Saliency tells us *where* to look; a descriptor decides *what* to record there, encoding the neighborhood as a vector so that the same part can be matched again on a new image. What makes one descriptor better than another is best understood as a balancing act among four competing demands, summarized in Table 12.

Property	What it means
Invariance	Stays stable under nuisance changes: translation, rotation, scale, and illumination.
Discriminability	Separates similar classes reliably; different parts get different descriptors.
Compactness	Stores enough information in a short vector to keep matching and storage cheap.
Computational cost	Remains feasible to extract and compare on large datasets.

Table 12: The four criteria for a good local feature. Invariance and discriminability pull in opposite directions, and compactness and cost bound how far either can be pushed.

These demands genuinely conflict. Pushing invariance to the limit deliberately discards information — if a descriptor is built to ignore rotation, it can no longer tell two parts apart that differ only by rotation — which eats into discriminability. And richer, more discriminative descriptors are longer and costlier to compute and compare. Every concrete descriptor is one particular compromise among the four.

The *histogram of oriented gradients* (HOG) is the canonical compromise on the shape-and-appearance side, and it is worth seeing how it is built because it reuses the gradients from Equation (28). The region is divided into a grid of small *cells*; within each cell the gradient orientations of the pixels are collected into a histogram (for example over nine orientation bins), each pixel contributing in proportion to its gradient magnitude. Cells are then grouped into overlapping *blocks*, and the histograms in a block are normalized together before being concatenated, over all blocks, into one long feature vector. Figure 18 shows the whole construction on a simple shape. Two design choices give HOG its character. Summarizing gradient *orientations* rather than raw intensities, together with the block normalization, makes it robust to moderate lighting changes and small deformations. But because the descriptor is laid out as a fixed grid of oriented histograms, rotating the patch permutes the bins and changes the vector: standard HOG is *not* rotation invariant. In return it turns the diffuse evidence of many edges into a single classifier-ready vector, which is why it remains a standard baseline against which learned features are measured.

Two further descriptor families round out the classical toolkit and contrast usefully with HOG. *Fourier descriptors* encode a closed contour by the Fourier coefficients of its boundary curve; translation, rotation, and scaling of the shape act on those coefficients in simple, separable ways (a shift of the constant term, a phase change, an overall magnitude scaling), so after normalization the coefficients become invariant shape descriptors. Their construction, normalization, and use for smoothing are worked out in Appendix C. *Hu moments* combine image moments into a handful of quantities that are likewise invariant to translation, scale, and rotation. For detecting the salient *points* in the first place, the standard keypoint operators (Harris, Förstner, SIFT) provide the geometric counterpart to the information-theoretic salient regions of the previous subsection. The common thread across all of them is the same: a descriptor converts a neighborhood into a compact vector, and closeness between two such vectors is taken to mean that the two patches show the same thing.

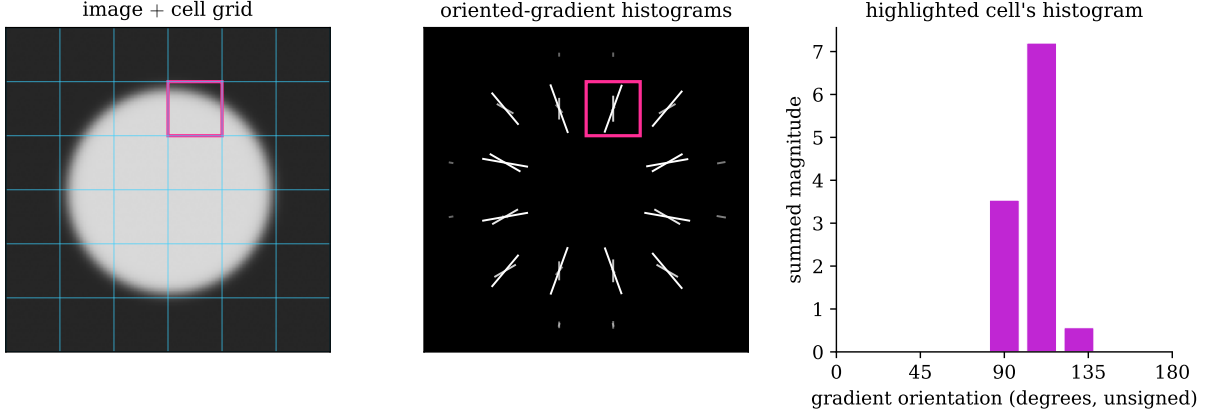


Figure 18: Building a HOG descriptor, computed on a synthetic disk. *Left*: the region is divided into a grid of cells (one highlighted in magenta). *Middle*: each cell is replaced by a small star of strokes, one per orientation bin, whose length is the total gradient magnitude falling in that orientation; since gradients run perpendicular to edges, the strokes point radially around the disk’s rim, while flat interior and exterior cells stay empty. *Right*: the highlighted cell’s nine-bin histogram of gradient orientation, each pixel weighted by its gradient magnitude, with a clear peak at the orientation of the local edge. Stacking every cell’s (block-normalized) histogram into one vector yields the descriptor.

3.5 Putting It Together: The Implicit Shape Model

The implicit shape model (ISM) is where the three threads of this section — voting, saliency, and description — finally meet. Its goal is object *categorization* in real scenes: given training images of a category, learn a model from the local appearance of its parts; given a new image, both label and segment the instances. The one-sentence summary is that ISM is a generalized Hough transform whose fixed geometric R-table has been replaced by a *learned appearance codebook*. Where the GHT indexed displacement vectors by an edge’s gradient orientation, ISM indexes them by a patch’s appearance; the voting machinery is otherwise the same.

The **training phase** builds that codebook. Salient points are extracted from the training images (using the salient-region or keypoint detectors above), and each one defines a *patch* at its characteristic scale. The patches are described by their local appearance and then grouped so that visually similar patches become a single *codebook* entry — a prototype part such as “front wheel” or “headlight.” The grouping is done by iterative merging: every patch starts as its own cluster, and at each step the two most similar clusters C_1 and C_2 are merged, provided their average pairwise similarity stays above a threshold t . Similarity between two patches p and q is the normalized greyscale correlation, and the similarity of two clusters averages it over all their members:

$$\text{NGC}(p, q) = \frac{\sum_i (p_i - \bar{p})(q_i - \bar{q})}{\sqrt{\sum_i (p_i - \bar{p})^2 \sum_i (q_i - \bar{q})^2}}, \quad \text{sim}(C_1, C_2) = \sum_{p \in C_1, q \in C_2} \frac{\text{NGC}(p, q)}{|C_1| |C_2|}. \quad (36)$$

In $\text{NGC}(p, q)$, the index i runs over the pixels of the patch, p_i and q_i are their grey values, and $\bar{p}, \bar{q}$ are the patch means; subtracting the means removes brightness offsets and the denominator removes contrast scaling, so NGC is a correlation coefficient between -1 and 1 that equals 1 for patches identical up to brightness and contrast and near 0 for unrelated ones. The cluster similarity $\text{sim}(C_1, C_2)$ is then just the average NGC over all cross-pairs, with $|C_i|$ the number of patches in cluster C_i . Stopping the merges at the threshold t keeps clusters compact and groups only genuinely similar patches. Finally, for each codebook entry we record, across all the training images, the offset vectors from every matching patch to the object center — the appearance

analogue of the R-table’s displacement vectors, specifying where on the object each prototype part tends to appear. In order, training runs through five steps:

1. extract salient points from the training images and cut out a patch at each point’s scale;
2. describe each patch by its local appearance (grey values or a keypoint descriptor);
3. cluster similar appearances by iterative merging and store the cluster centers as the codebook;
4. match the codebook entries against all training patches and activate every entry whose similarity exceeds t ;
5. for every activation, store the offset vector from the patch to the object center with the activated entry.

The codebook itself thus holds *prototypes of object parts* with a stable appearance across images; the offsets to the reference point are attached to each entry afterwards. They play the role of the R-table rows, not of the codebook’s content.

The **prediction phase** runs the voting. Salient patches are extracted from the test image and matched against the codebook; an entry is *activated* whenever its NGC similarity to a patch exceeds the threshold t . Each activated entry then casts generalized Hough votes for the object center, placing a vote at the patch position plus each of that entry’s stored offsets — exactly the GHT step, but driven by appearance rather than edge orientation. Unlike the discrete accumulator of the classical GHT, ISM votes in a *continuous* space, and the densest cluster of votes is located with mean-shift mode estimation (equivalently, a non-parametric density estimate); that mode is the hypothesized object center. A final *back-projection* step reverses the voting: the patches that supported the winning center are projected back into the image, which both localizes the object within a surrounding region and, through a probabilistic formulation, yields a pixel-level *segmentation* rather than a mere bounding box. Figure 22 lays out the whole pipeline, training above and prediction below.

Because ISM grows directly out of the generalized Hough transform, it helps to see the two side by side; Table 13 lines up what changes and what stays the same.

Aspect	Generalized Hough transform	Implicit shape model
Model of a part	Fixed geometric R-table	Learned appearance codebook
Local evidence	Edge + gradient orientation	Salient patch + appearance
Indexing	Gradient-orientation row	Codebook match above threshold t
Voting space	Discrete accumulator	Continuous (mean-shift modes)
Output	Center / pose of one known shape	Category detection <i>and</i> segmentation

Table 13: From a single known shape to an object category: ISM keeps the voting machinery of the generalized Hough transform but learns the parts and their spatial distribution from data.

Two further models slot naturally into the same picture. *Hough forests* replace the codebook-and-matching step with a random forest that maps each patch directly to a distribution of votes, learning the appearance grouping and the offsets together in one trained predictor. *Deformable part models* (DPM) take a complementary route: an object is a coarse root HOG template plus a few higher-resolution part templates attached to it by spring-like deformation costs, the whole thing scored with a latent-variable classifier. The Hough forest still votes, but learns the votes instead of looking them up in a codebook. DPM does not vote at all: it localizes objects by

classifying windows of the HOG feature map at many positions and scales. Appendix D gives the details needed for both. Both keep the central bargain of this section — local parts proposing global configurations — while changing how the parts are represented and how their votes are combined. The implicit shape model itself is due to Leibe, Leonardis, and Schiele (ECCV’04 workshop; IJCV 2008).

It is worth ending where we began. Every model in this section is one instance of the same mechanism, differing only in what counts as local evidence and how much flexibility the global hypothesis is allowed; Table 14 lines them up along that single axis.

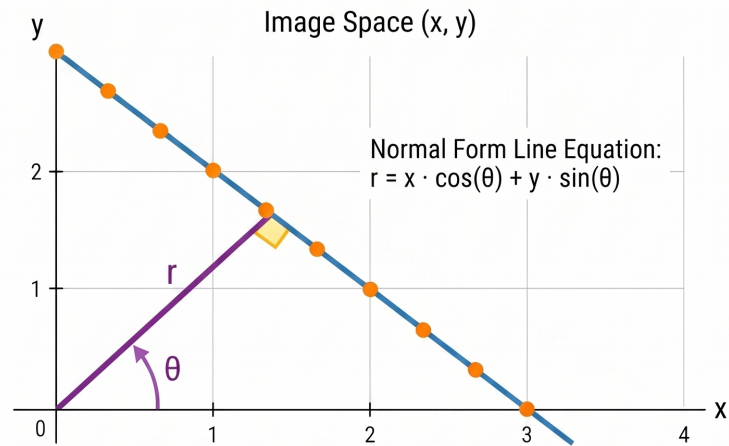
Model	What casts a vote	What it votes for
Hough line detector	An oriented edge pixel	The line (r, θ) it lies on
Generalized Hough transform	An edge, indexed by gradient orientation	The reference center of a known shape, in (x, y, s, θ)
Implicit shape model	A patch matched to a code-book entry	The object center, in a continuous space, then a segmentation

Table 14: The through-line of the section. As the local evidence grows richer — from edge to oriented edge to learned appearance patch — the same voting scheme stretches from a single line to an arbitrary shape to a whole object category.

A Figures: Visual Object Models

The Hough Transform: Detecting Straight Lines

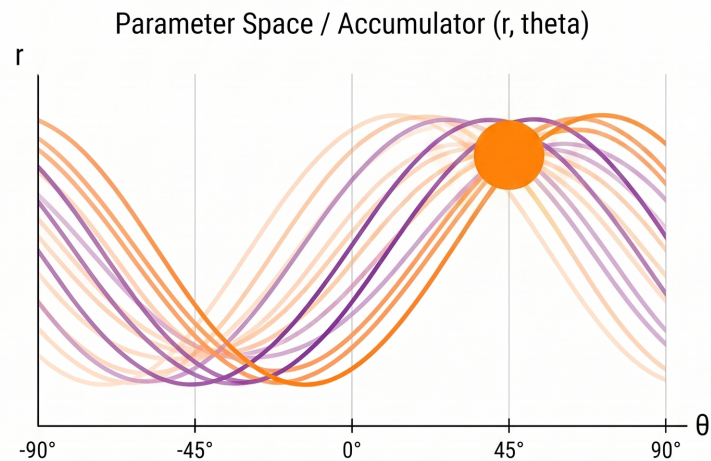
The Hough Transform identifies shapes by mapping image pixels into a parameter space. For straight lines, each pixel votes for all possible lines it could belong to, represented as sinusoidal curves that intersect at a single coordinate (r, θ) corresponding to the detected line.



Definition: A line is defined by its perpendicular distance r and its angle θ from the origin.

Mapping Edge Pixels: Collinear points on a line share a constant (r, θ) relationship used for detection.

Geometry: r is the shortest distance to the line, meeting it at a right-angle foot.



Pixel-to-Curve Transformation: Every image pixel transforms into a unique sinusoidal curve in the (r, θ) plane.

Accumulator Voting: Collinear points result in curves that intersect at one specific coordinate.

Peak Detection: A pronounced maximum in the accumulator triggers the detection of a straight line.

NotebookLM

Figure 19: Hough duality for line detection. Collinear edge points in image space (top) all map to the same point in the (r, θ) accumulator (bottom); each individual point maps to a sinusoid, and the sinusoids intersect where the pixels agree. The pronounced peak triggers the detection.

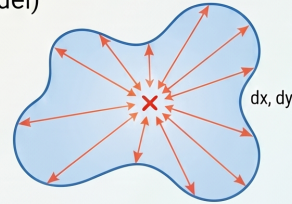
The Generalized Hough Transform: A Two-Stage Process for Shape Detection

Enables detection of arbitrary 2D shapes by mapping edge orientations to displacement vectors through a relational model (R-Table) and probabilistic voting.

Stage 1: Training (Building the Model)

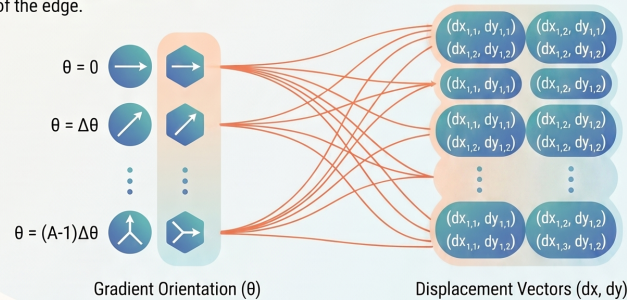
Define Reference Point and Displacement

Select a center point for the arbitrary contour and map displacement vectors from every edge point.



Construct the R-Table

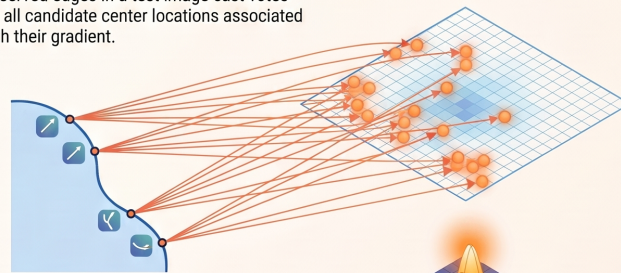
Store displacement vectors (dx, dy) in a table indexed by the local gradient orientation of the edge.



Stage 2: Detection (Object Recognition)

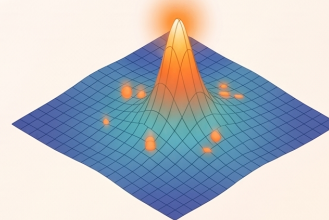
Probabilistic Voting

Observed edges in a test image cast votes for all candidate center locations associated with their gradient.



Accumulator Peak Identification

The true center of the object is identified where the highest density of votes (the pronounced peak) occurs.

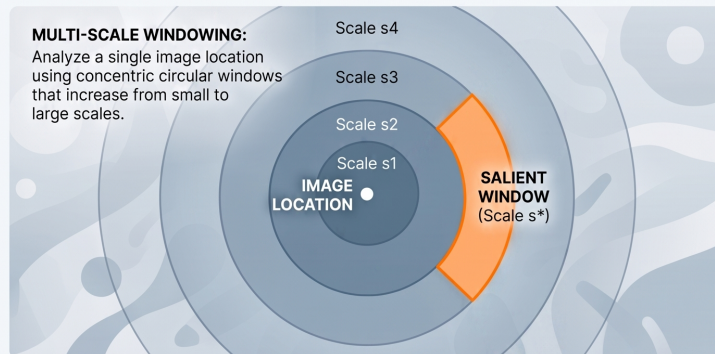


NotebookLM

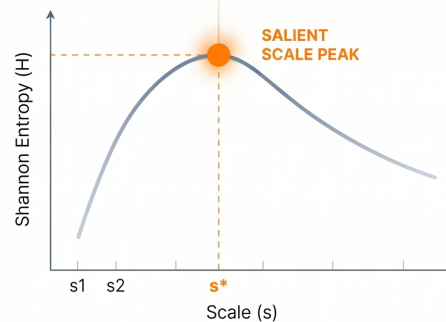
Figure 20: Generalized Hough transform. Each model edge stores a displacement vector to the reference point, organized in the R-table by gradient orientation. Because the same orientation occurs at several places on the shape, one observed edge casts votes for several candidate centers; the true center is where votes accumulate.

Detecting Saliency: The Kadir-Brady Scale-Space Model

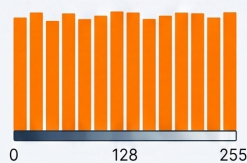
Saliency is a measure of "rareness" or local complexity. This model identifies salient points by analyzing the Shannon Entropy of local image attributes across a range of scales (window sizes) to find where visual information is most distinctive.



THE ENTROPY CURVE $H(s)$:
Calculate local Shannon Entropy at each scale to produce a plot of entropy as a function of scale.
The salient scale is defined as the point where the entropy-versus-scale curve reaches a clear maximum.



FLAT HISTOGRAM
(Evenly Distributed)



HIGH ENTROPY / SALIENCY

A flat local intensity histogram indicates high entropy, signifying a rare and complex image region.

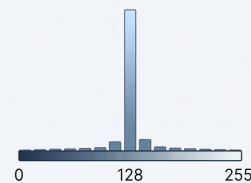
SALIENCY AS LOCAL COMPLEXITY:

High entropy corresponds to a measure of uncertainty, randomness, or average information content.

MEASURE OF SELF-UNSIMILARITY:

Selecting entropy peaks ensures saliency is valid in both the spatial domain and scale space.

PEAKED HISTOGRAM
(Low Uncertainty)



LOW ENTROPY

A peaked histogram results in low entropy, corresponding to low information content and low saliency.

NotebookLM

Figure 21: Scale-space saliency. The same location is analyzed over a range of window sizes; entropy as a function of scale has a peak at the salient scale. A flat local histogram (high entropy) signals a rare, informative region, while the scale-localization weight discounts patches whose statistics barely change across scale.

The Implicit Shape Model (ISM) Pipeline: From Local Patches to Global Detection

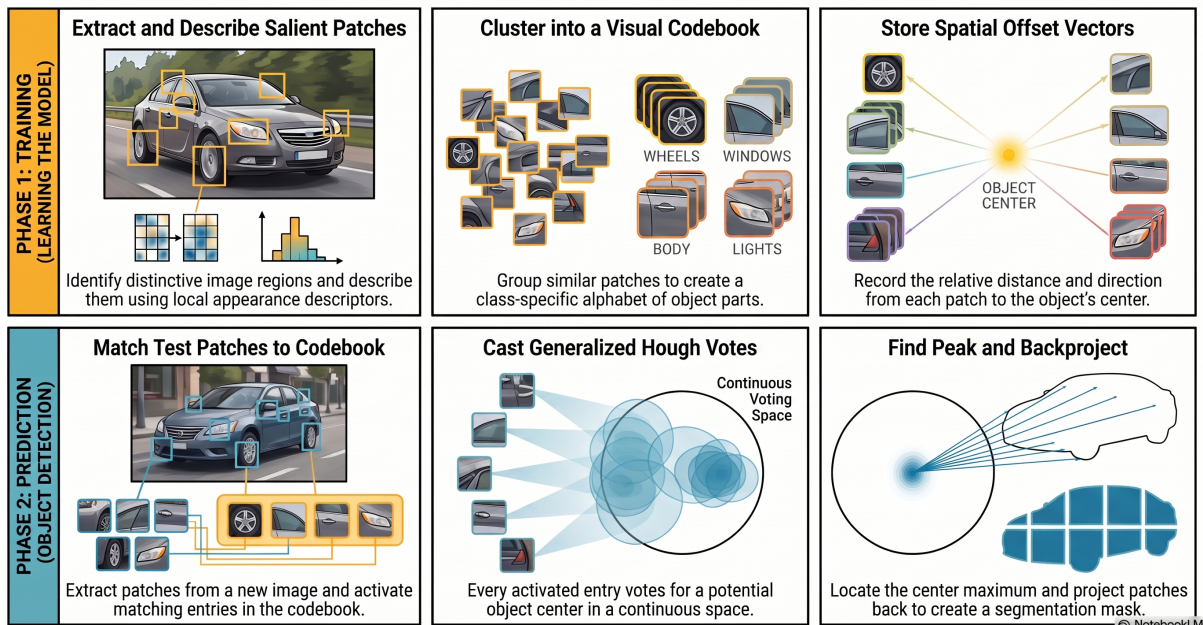


Figure 22: The implicit shape model. Training (top row): salient patches are described, clustered into a codebook, and annotated with offset vectors to the object center. Prediction (bottom row): test patches are matched to the codebook, cast generalized Hough votes for the center, and the peak is back-projected to recover and segment the object.

B Edge Detection Essentials

Every voting method in Section 3 starts from edge pixels and their gradients. The intuition is simple: an edge is where the brightness changes abruptly, so its location is where the first derivative of the image is large. This appendix collects what is needed to compute gradients and to understand the reference edge detector, Canny.

Gradients from derivative filters. On a pixel grid the derivatives $G_x = \partial I / \partial x$ and $G_y = \partial I / \partial y$ are approximated by small convolution kernels. The Sobel kernels combine a central difference in one direction with a smoothing in the other:

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad S_y = S_x^\top = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}. \quad (37)$$

The Prewitt kernels are the same with the weights 2 replaced by 1. From the two responses follow the gradient magnitude $\|\nabla I\| = \sqrt{G_x^2 + G_y^2}$ (edge strength) and the orientation $\theta = \text{atan2}(G_y, G_x)$. The gradient points from dark to bright and is perpendicular to the edge; the edge itself runs along $\theta \pm 90^\circ$.

The Canny detector. Thresholding the gradient magnitude alone gives thick, broken, noisy edges. Canny fixes this in four steps:

1. smooth the image with a Gaussian to suppress noise;
2. compute gradient magnitude and orientation, e.g. with Sobel kernels;
3. *non-maximum suppression*: keep a pixel only if its magnitude is a local maximum along the gradient direction, which thins every edge to a one-pixel-wide line;
4. *hysteresis thresholding* with a high and a low threshold: pixels above the high threshold are strong edges and are kept; pixels between the thresholds are weak edges and are kept only if they connect to a strong edge; everything below the low threshold is discarded.

Hysteresis removes isolated noise responses while preserving long, coherent boundaries. The detector fails where the image offers no contrast: low-contrast boundaries, haze, and gradual transitions stay below threshold, and strong texture produces fragmented contours. The smoothing width and the two thresholds must be tuned to the imaging conditions.

C Fourier Transform and Fourier Descriptors

The Fourier transform rewrites a signal as a sum of sinusoids of different frequencies; the list of their complex weights is the *spectrum*. Low frequencies describe the coarse shape of a signal, high frequencies its fine detail and noise. Two applications appear in object models: correlating gradient images quickly as an alternative to R-tables, and describing a closed contour by the spectrum of its boundary (Fourier descriptors). Both need only the facts collected here.

C.1 The Discrete Fourier Transform

For a discrete signal $x(0), \dots, x(N-1)$ the discrete Fourier transform (DFT) and its inverse are

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-2\pi i k n / N}, \quad x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{2\pi i k n / N}. \quad (38)$$

The spectrum has as many coefficients as the signal has samples, and it is periodic, $X(k) = X(k + N)$, so the upper half of the indices can equally be read as negative frequencies: $X(N - 1) = X(-1)$, $X(N - 2) = X(-2)$, and so on. Implementations therefore often store the spectrum as $[X(0), X(1), \dots, X(N/2), X(-N/2), \dots, X(-2), X(-1)]$. The zero-frequency term $X(0) = \sum_n x(n)$ is N times the mean of the signal.

Real signals. If x is real-valued, the spectrum is *Hermitian*: $X(-k) = X(k)^*$, where $*$ is complex conjugation. Half of the spectrum then determines the other half. For example, if a real signal with $N = 17$ samples has $X(0), \dots, X(8) = [10, -5 + i, -0.5 + 0.2i, -0.04 + 0.03i, 0, 0, 0, 0, 0]$, the remaining entries $X(-8), \dots, X(-1)$ are $[0, 0, 0, 0, 0, -0.04 - 0.03i, -0.5 - 0.2i, -5 - i]$: conjugate and mirror, without changing the sign of the real parts.

Standard spectra. Table 15 lists the transform pairs used most often.

Signal	Spectrum
$\cos(2\pi f_0 x)$	two real spikes at $\pm f_0$
$\sin(2\pi f_0 x)$	two imaginary spikes at $\pm f_0$ with opposite signs
rectangle of width w	sinc function, $\propto \text{sinc}(wf)$; a narrower rectangle gives a wider sinc
sinc function	rectangle (ideal low-pass)
Gaussian	Gaussian; a narrow Gaussian gives a wide one

Table 15: Common Fourier pairs. Width in the signal domain and width in the frequency domain are inversely related.

Convolution and correlation theorems. Convolution in the signal domain is multiplication in the frequency domain, and correlation is the same with *one* factor conjugated:

$$f * g = \text{DFT}^{-1}(\text{DFT}(f) \cdot \text{DFT}(g)), \quad f \star g = \text{DFT}^{-1}(\text{DFT}(f)^* \cdot \text{DFT}(g)). \quad (39)$$

Conjugating both factors, or neither, does not give the correlation. The practical value is speed: a correlation of a template with a whole image costs two forward DFTs, a pointwise product, and one inverse DFT.

Gradient correlation instead of R-tables. Write the gradients of the image I and of the template O as complex numbers,

$$I_I = \frac{\partial I}{\partial x} + i \frac{\partial I}{\partial y}, \quad O_I = \frac{\partial O}{\partial x} + i \frac{\partial O}{\partial y}. \quad (40)$$

For two complex numbers $a = a_x + ia_y$ and $b = b_x + ib_y$ the product $a^*b = (a_x b_x + a_y b_y) + i(a_x b_y - a_y b_x)$ has the dot product of the two gradient vectors as its real part. Hence $\text{Re}(I_I \star O_I)(x, y)$ sums, over the whole template placed at (x, y) , the dot products between template gradients and image gradients. It is large where many image edges have the position and orientation the template expects, which is almost the same as Hough voting with an R-table. The imaginary part contains cross products instead and is not the voting score. Equation (39) computes the correlation for all positions at once.

C.2 Fourier Descriptors

A closed contour can be described as a periodic complex signal. Trace the boundary of a segmented object at N points and write each point as one complex number,

$$u(n) = x(n) + i y(n), \quad n = 0, \dots, N-1. \quad (41)$$

The Fourier descriptor is its DFT, $F(\mu) = \text{DFT}(u)(\mu)$ for $\mu = 0, \dots, N-1$. From a greyscale image the pipeline is therefore: segment the object, trace its boundary at N points, form u , apply the DFT, normalize, and keep the low-frequency coefficients. Two counting facts follow directly from the DFT. The descriptor has exactly as many coefficients as the contour has points, so a descriptor $F(\mu)$ with $\mu \in \{0, \dots, 69\}$ describes a contour of 70 points, whatever the value of $F(0)$. And $F(0) = \sum_n u(n) = N \cdot (\bar{x} + i \bar{y})$ is N times the centroid: for a contour of $N = 20$ points centered at $(10, 20)$, $F(0) = 200 + 400i$ (with a $1/N$ -normalized DFT it would be the centroid $10 + 20i$ itself).

How transformations act. Each geometric change of the shape has a simple effect on the spectrum, which is what makes normalization possible (Table 16). A translation by $t = t_x + it_y$ adds t to every $u(n)$ and therefore only changes $F(0)$ (by Nt). A scaling by a multiplies every coefficient by a . A rotation by φ multiplies every coefficient by $e^{i\varphi}$, a pure phase change. A different start point on the contour multiplies $F(\mu)$ by a phase $e^{2\pi i \mu m/N}$ that grows with μ .

Invariance to	Effect on F	Normalization
Translation	changes only $F(0)$	set $\tilde{F}(0) := 0$
Scale	multiplies all $F(\mu)$ by a	divide by $ F(k) $ for a fixed $k \neq 0$, e.g. $k = 1$
Rotation	multiplies all $F(\mu)$ by $e^{i\varphi}$	keep magnitudes $ F(\mu) $ (or normalize the phase)
Start point	phase $e^{2\pi i \mu m/N}$	keep magnitudes $ F(\mu) $

Table 16: Normalizing a Fourier descriptor. Each step removes one nuisance transformation; mirroring and noise are not removed by any of them.

The normalization $\tilde{F}(0) := 0$, $\tilde{F}(\mu) := F(\mu)/|F(2)|$ therefore makes the descriptor invariant to translation (first step) and to scale (second step, with $k = 2$). It is not invariant to rotation, because the phases are kept. It is not invariant to mirroring, since mirroring conjugates u and exchanges positive and negative frequencies. Noise is never removed by normalization. Plots of normalized descriptors can be recognized by the same rules: the zero-frequency value is zero after translation normalization, the reference coefficient has magnitude 1 after scale normalization, and all values are real and non-negative after taking magnitudes.

Smoothing a shape. Fine detail and noise of the contour live in the high frequencies, which are the indices in the middle of the stored array (large $|\mu|$). A shape is smoothed by a low-pass operation: set those coefficients to zero or attenuate them, keep the low frequencies at both ends of the array, and apply the inverse DFT. Keeping only magnitudes destroys the shape, because the phases carry the positions of the points. Dividing the whole spectrum by a constant only rescales the shape and does not smooth it.

Limitations. Fourier descriptors need a clean segmentation and one closed, connected contour. They are global: a partial occlusion, or two touching objects, changes the whole contour and therefore every coefficient. The number of contour points and the sampling must be consistent across shapes to compare descriptors.

D Classical Classifiers and Part-Based Detectors

The classification stage of the processing chain in Figure 6 can be filled by many learners. Part-based detectors build on them. This appendix covers the classifiers and two detectors that exercises ask about beyond the implicit shape model: the Hough forest and the deformable part model. For each, the aim is to know its working principle and what exactly it outputs.

Linear classifiers and SVMs. A linear classifier decides by the sign of an affine function, $g(x) = w^\top x + b$; its decision boundary is a hyperplane. For linearly separable data many hyperplanes classify the training set correctly. A *linear support vector machine* (SVM) picks the one with the *maximum margin*: the hyperplane whose distance to the nearest training samples of both classes is largest, running through the middle of the gap. Only these nearest samples, the support vectors, determine it. A soft-margin variant tolerates some misclassified samples, and kernels allow nonlinear boundaries. In a figure with several separating lines, the SVM line is the one that keeps equal and largest distance to both classes, not one that touches a sample.

Random forests. A decision tree routes a sample through binary node tests (for example “is feature j larger than τ ?”) to a leaf that stores a class distribution. During training, each node picks, from a set of randomly generated candidate tests, the one that best reduces the class entropy of the samples it splits (information gain). A random forest trains many trees, each on a random subset of the data and with random candidate tests, and averages the leaf distributions of all trees. The randomness makes the trees differ, so their average has lower variance than any single tree.

Hough forests. A Hough forest is a random forest whose trees vote like the implicit shape model. Training samples are image patches, each labeled as object or background and, for object patches, with the offset vector to the object center. The node tests are simple and fast: compare the values of one feature channel (intensity, gradient, and so on) at two pixel positions inside the patch, $f(p) < f(q) + \tau$. Each node is optimized for one of two energies, chosen at random: the *class-label uncertainty* (entropy of object versus background) or the *offset uncertainty* (variance of the offset vectors of the object patches). Each leaf stores the fraction of object patches that reached it (a class probability) and the list of their offset vectors. At detection time every patch of the image descends all trees, and each leaf casts its offsets as votes for the object center, weighted by its class probability. The codebook and its matching step are thus replaced by a learned mapping from appearance to votes.

Deformable part models. A deformable part model (DPM) describes an object by a coarse *root* filter on HOG features, several *part* filters at twice the resolution, and a quadratic deformation cost for each part’s displacement from its anchor position. The score of a hypothesis with root position p_0 is

$$\text{score}(p_0) = F_0 \cdot \phi(p_0) + \sum_{i=1}^n \max_{p_i} \left[F_i \cdot \phi(p_i) - d_i \cdot (dx_i, dy_i, dx_i^2, dy_i^2) \right] + b, \quad (42)$$

where $\phi(p)$ is the HOG feature window at position p , F_i are the filter weights, and (dx_i, dy_i) is part i ’s displacement from its anchor. The part positions are latent variables, and the filters are trained with a latent SVM. Localization is a sliding-window search: the score is evaluated at every position and scale of a HOG feature pyramid, and the high-scoring windows, after non-maxima suppression, are the detections. The SVM only outputs a score for a given window, not a location. Objects are found by classifying windows at many locations, not by a codebook and Hough voting.